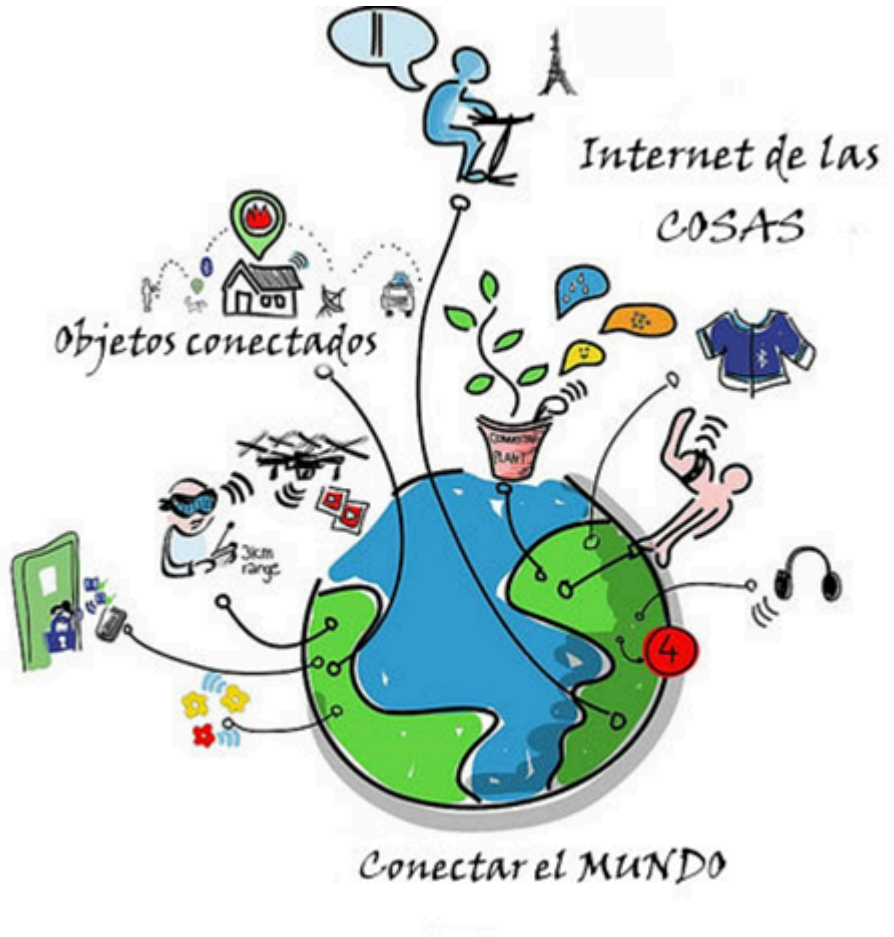
Internet of Thinks (IoT)

¿Qué es IoT?

Se le llama **Internet de las cosas**, en inglés **Internet of Things (IoT)** a la posibilidad de interconexión y transmisión de datos entre objetos cotidianos e internet.



Los aparatos electrónicos y los dispositivos digitales cotidianos tienen circuitos y sensores que les permite recolectar y compartir datos con la internet sin la intervención de personas.

¿Cómo funciona el IoT?

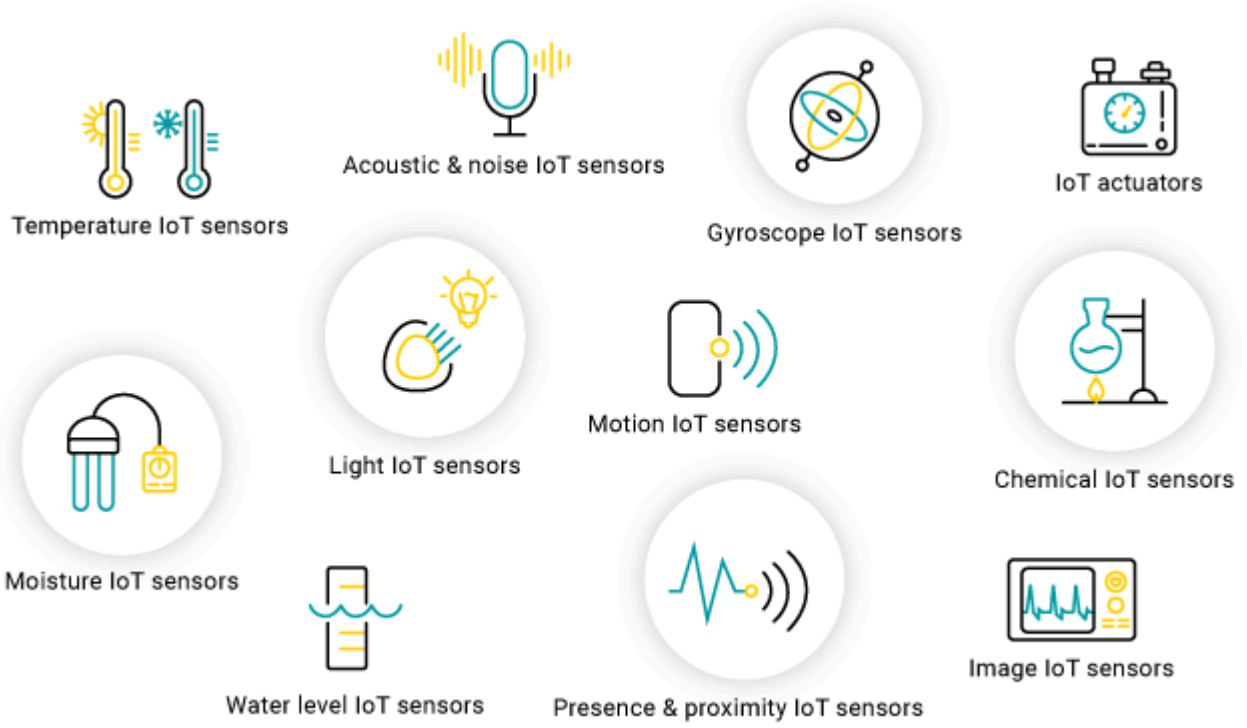
Los **dispositivos IoT** se conectan con un proceso llamado M2M (machine to machine, o máquina a máquina) en el que dos dispositivos o máquinas cualesquiera se comunican entre sí utilizando cualquier tipo de conectividad (que puede ser cable, WiFi, Bluetooth, etc.), haciendo su trabajo sin la necesidad de que un humano intervenga. No deja de ser el mismo concepto que las conexiones Peer to Peer de los ordenadores para jugar online o compartir archivos.

Entonces nos encontramos que un dispositivo IoT recolecta, procesa y analiza una gran cantidad de información, entre la que se incluye información sobre los hábitos y preferencias de consumo de cada uno de los usuarios, así como por ejemplo poder gestionar su salud u otras utilidades para el día a día que no serían posibles si. Como, por ejemplo, si hay algún fallo en tu coche, el ordenador a bordo del mismo puede enviar un aviso para que lo lleves al taller.

Un ejemplo de dispositivo IoT es el altavoz inteligente Amazon de Alexa, el cual se encuentra conectado a la red de redes para obtener información e interactúa con otros dispositivos para darnos la capacidad de controlarlos a través de simples comandos de voz.

Dispositivos IoT

Un dispositivo IoT consiste en un **objeto** al que se le ha dotado de conexión a Internet y cierta inteligencia software, sobre el que se pueden medir parámetros físicos (mediante **sensores**) o actuar remotamente (**actuadores**) y que por tanto permite generar un ecosistema de servicios alrededor del mismo.



Para lograr que sensores y actuadores recolecten y envíen datos del mundo físico a Internet, los dispositivos IoT cuentan con un componente escencial: el microcontrolador. Los microcontroladores son **Sistemas Embebidos de Control** diseñados para realizar funciones dedicadas, normalmente en tiempo real. Al contrario de lo que ocurre con los ordenadores de propósito general, que están diseñados para cubrir un amplio rango de necesidades, los sistemas embebidos se diseñan para cubrir necesidades específicas y por tanto forman parte de multitud de productos en los que se requiere una cierta potencia de proceso como electrodomésticos, vehículos, máquinas, etc.

Sistemas embebidos

Formalmente, un sistema embebido se construye en torno a un microcontrolador (MCU) o un microprocesador, que es un circuito integrado que contiene los elementos esenciales para ejecutar un programa y controlar un sistema. Sin embargo, su verdadera naturaleza se define por un conjunto de características que lo diferencian de la computación tradicional :

- Recursos Limitados: Los sistemas embebidos están diseñados para ser eficientes en coste y consumo energético. Esto se traduce en una capacidad de procesamiento (CPU), memoria (RAM/Flash) y almacenamiento muy restringido, lo que obliga a los diseñadores a omitir capas de seguridad que se dan por sentadas en otros sistemas.

- Ciclos de Vida Largos: Muchos dispositivos embebidos, especialmente en entornos industriales o de infraestructura, están diseñados para operar durante décadas. Este largo ciclo de vida significa que a menudo funcionan con software y hardware obsoletos, sin recibir actualizaciones, acumulando un déficit de seguridad que crece con el tiempo, conocido como deuda de seguridad.
- Exposición Física: Al estar integrados en el mundo físico, muchos de estos dispositivos son accesibles a un atacante. Esta exposición permite la manipulación directa del hardware y la extracción de información, lo que habilita una clase entera de ataques a nivel de hardware que son inviables en entornos de computación tradicionales.

Ventajas del IoT

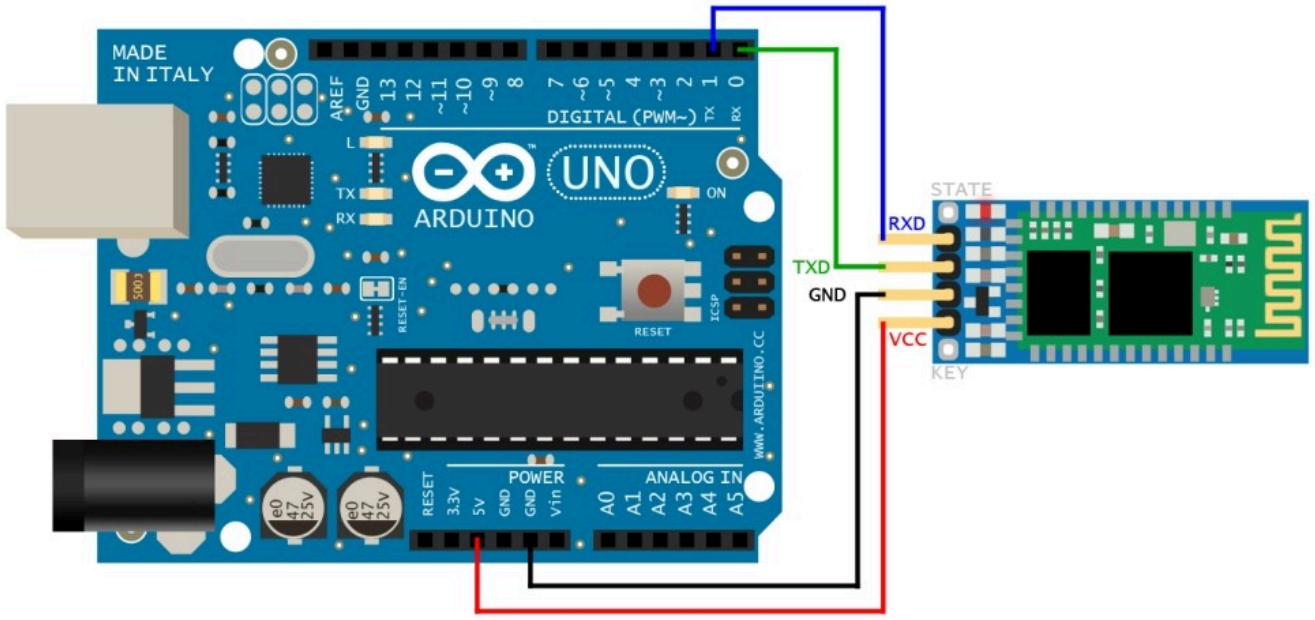
Algunas de las ventajas mas significativas del IoT:

- **Conectividad y Comunicación**: El IoT permite la conexión de dispositivos y sistemas en red, lo que facilita la comunicación y la transferencia de datos en tiempo real entre ellos.
- **Automatización y Eficiencia**: Los dispositivos IoT pueden recopilar datos, analizarlos y tomar decisiones automáticas, lo que lleva a una mayor eficiencia operativa y a la automatización de tareas.
- **Monitoreo Remoto**: El IoT permite el monitoreo y control remoto de dispositivos y sistemas, lo que facilita la supervisión y gestión de procesos desde cualquier ubicación.
- **Mejora de la Productividad**: Al automatizar tareas repetitivas y optimizar procesos, el IoT puede mejorar la productividad en diversas industrias y actividades.
- **Personalización y Experiencia del Usuario**: El IoT permite la personalización de servicios y experiencias para los usuarios, adaptándose a sus necesidades y preferencias individuales.
- **Toma de Decisiones Basada en Datos**: Los datos recopilados por los dispositivos IoT proporcionan información valiosa para tomar decisiones informadas, lo que lleva a una toma de decisiones más precisa y efectiva.

Microcontroladores ARDUINO

Arduino es una plataforma de creación de electrónica de código abierto, la cual está basada en hardware y software libre, flexible y fácil de utilizar para los creadores y desarrolladores. Esta plataforma permite crear diferentes tipos de microordenadores de una sola placa a los que la comunidad de creadores puede darles diferentes tipos de uso.

Para poder entender este concepto, primero vas a tener que entender los conceptos de hardware libre y el software libre. El hardware libre son los dispositivos cuyas especificaciones y diagramas son de acceso público, de manera que cualquiera puede replicarlos. Esto quiere decir que Arduino ofrece las bases para que cualquier otra persona o empresa pueda crear sus propias placas, pudiendo ser diferentes entre ellas pero igualmente funcionales al partir de la misma base.



El **Arduino** es una placa basada en un microcontrolador ATMEL. Los microcontroladores son circuitos integrados en los que se pueden grabar instrucciones, las cuales las escribes con el lenguaje de programación que puedes utilizar en el entorno Arduino IDE. Estas instrucciones permiten crear programas que interactúan con los circuitos de la placa.

El microcontrolador de Arduino posee lo que se llama una interfaz de entrada, que es una conexión en la que podemos conectar en la placa diferentes tipos de periféricos. La información de estos periféricos que conectes se trasladará al microcontrolador, el cual se encargará de procesar los datos que le lleguen a través de ellos.

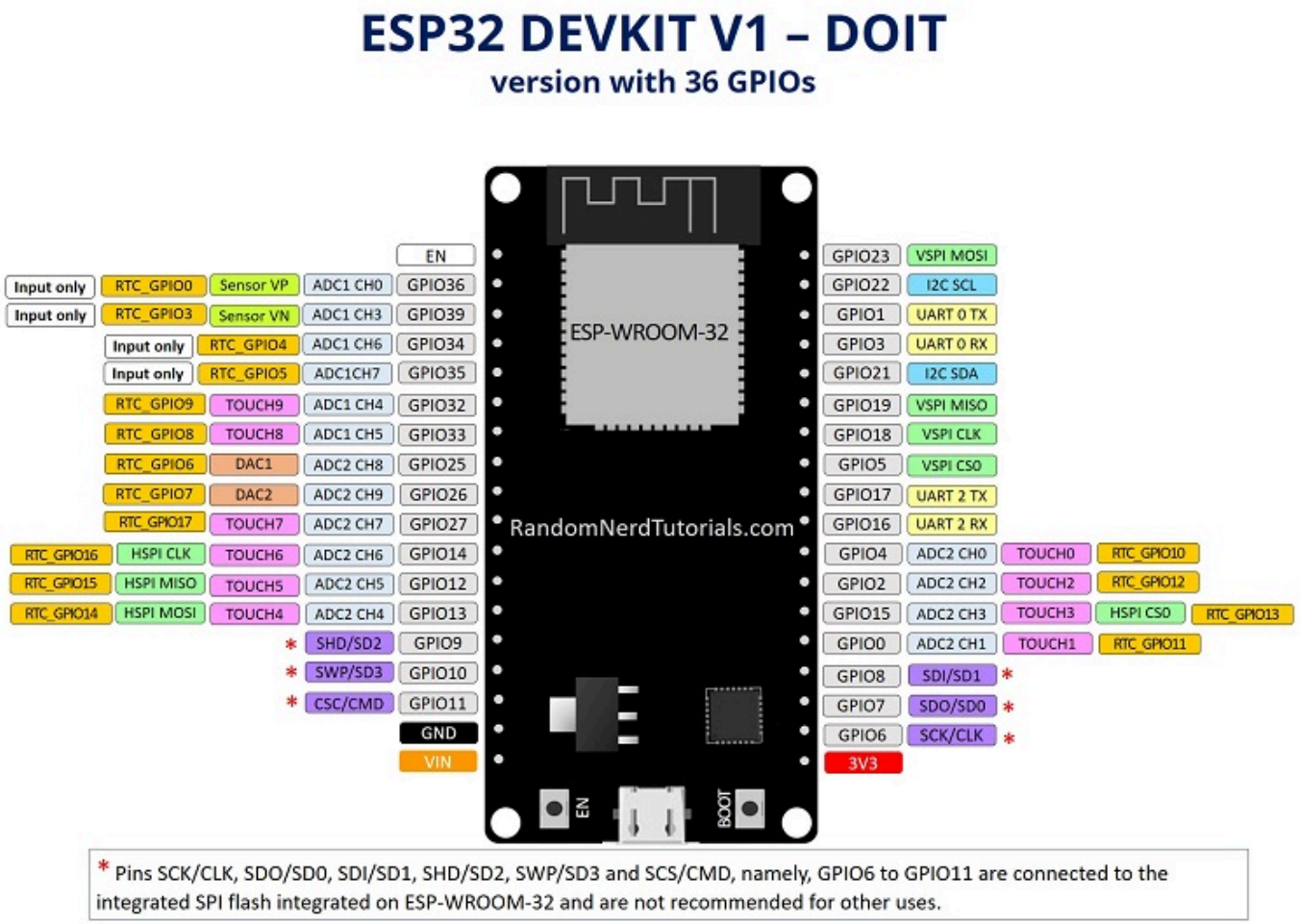
El tipo de periféricos que puedas utilizar para enviar datos al microcontrolador depende en gran medida de qué uso le estés pensando dar. Pueden ser cámaras para obtener imágenes, teclados para introducir datos, o diferentes tipos de sensores.

También cuenta con una interfaz de salida, que es la que se encarga de llevar la información que se ha procesado en el Arduino a otros periféricos. Estos periféricos pueden ser pantallas o altavoces en los que reproducir los datos procesados, pero también pueden ser otras placas o controladores.

Placas Esp32

ESP32 es una familia de microcontroladores de la empresa **Espressif Systems**. Su analogía más clara es la de un Arduino esteroides, que incluye Wifi, Bluetooth, altas velocidades de procesamiento, mayor capacidad de almacenamiento de datos, entre otras potentes características extra.

ESP32 es un microcontrolador de bajo costo y bajo consumo de energía conocido bajo la categoría de *SoC* (System on Chip)



Para mayor información consultar la hoja de datos (datasheet) del fabricante aquí: https://www.electrosoftcloud.com/wp-content/uploads/2021/04/esp32-wroom-32_datasheet_en.pdf.

Arquitectura del ESP32

La ESP32 es un System On Chip (SoC) diseñado por Espressif System, orientado a aplicaciones de sistemas embebidos, IoT y sistemas de control. Integra en un único chip capacidades de cómputo, conectividad inalámbrica, periféricos digitales y analógicos, y mecanismos de seguridad. Es el microcontrolador elegido para esta parte del curso.

Desde el punto de vista arquitectónico, la ESP32 es, Un SoC altamente integrado, Optimizado para conectividad, control y tiempo real blando, Capaz de ejecutar desde bare-metal hasta sistemas operativos de tiempo real, y diseñado para equilibrar potencia de cómputo, consumo energético y seguridad. Esto la posiciona como un puente entre los microcontroladores clásicos y los sistemas embebidos con Linux, manteniendo un fuerte control sobre el hardware y el tiempo de ejecución.

A continuación, describiremos su arquitectura.

1. Núcleo de procesamiento

La ESP32 integra uno o dos núcleos Xtensa LX6 de 32 bits, dependiendo de la variante específica del chip.

- Arquitectura Harvard modificada
- Pipeline de varias etapas
- Soporte para interrupciones anidadas

- Unidad de manejo de excepciones
- Soporte para instrucciones de multiplicación y división en hardware

Los núcleos pueden operar de manera:

- Simétrica (SMP) bajo un sistema operativo como FreeRTOS
- Asimétrica, asignando tareas específicas a cada core (por ejemplo, uno para comunicaciones y otro para aplicación)

Esto permite separar tareas de tiempo real de procesos de mayor carga computacional.

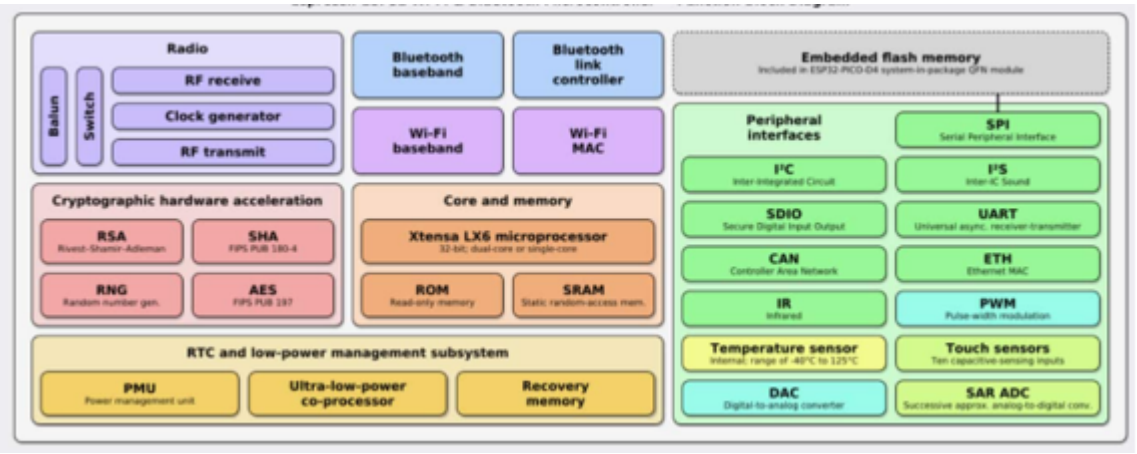


Fig. 2 Arquitectura de la ESP32

2. Subsistema de memoria

La arquitectura de memoria de la ESP32 combina memorias internas y externas:

2.1 Memorias internas

- ROM: Contiene el bootloader primario, rutinas básicas de inicialización y funciones de bajo nivel.
- SRAM: Utilizada para almacenar código ejecutable, datos, stack y heap.

2.2 Memoria Flash embebida o externa Contiene:

- Firmware de aplicación
- Sistema operativo
- Sistema de archivos (SPIFFS, FAT, LittleFS)

2.3 Memoria de recuperación

- Área destinada a retener información crítica ante reinicios o fallos, útil para crash recovery y estados persistentes.

3. Subsistema de conectividad inalámbrica

Uno de los aspectos centrales de la ESP32 es la integración completa de conectividad RF.

- Wi-Fi
- Bluetooth Este bloque permite comunicaciones inalámbricas sin necesidad de chips adicionales, reduciendo consumo y complejidad del sistema.

4. Criptografía y seguridad hardware

La ESP32 incorpora aceleradores criptográficos por hardware, diseñados para reducir la carga de CPU y mejorar la seguridad:

- AES (cifrado simétrico)
- SHA (hash criptográfico)
- RSA (criptografía asimétrica)
- RNG (generador de números aleatorios por hardware)

Estos módulos se utilizan para:

- Cifrado de comunicaciones
- Arranque seguro (secure boot)
- Encriptación de firmware y almacenamiento
- Protocolos TLS/SSL

5. Periféricos de comunicación digital

La ESP32 ofrece una amplia variedad de interfaces integradas:

- SPI
- I^2C
- I^2S (audio digital)
- SDIO
- UART
- TWAI® (CAN 2.0)
- Ethernet (MAC)
- IR (infrarrojo)

Estos periféricos están altamente configurables y pueden mapearse a distintos pines mediante la GPIO Matrix, característica clave del chip.

6. Periféricos de control y señal mixta

- Temporización y control: PWM, Timers de propósito general, watchdogs.
- Conversión analógica: ADC (convertidor analógico–digital) y DAC (convertidor digital–analógico)
- Sensores integrados: Touch sensors capacitivos, utilizados para interfaces táctiles sin componentes externos.

7. Gestión de energía y bajo consumo

La arquitectura incluye bloques dedicados a eficiencia energética:

- PMU (Power Management Unit): Controla modos de energía, reguladores internos y transiciones de estado.
- RTC (Real-Time Clock): Funciona de manera independiente al dominio principal de energía.
- ULP coprocessor (Ultra Low Power): Microcontrolador auxiliar capaz de ejecutar código simple, leer sensores y despertar al sistema principal con un consumo extremadamente bajo.

Tensión de trabajo

El voltaje de funcionamiento de los microprocesadores ESP es de **3,3 V** en comparación con el voltaje de funcionamiento de Arduino de **5 V**. Si las placas se utilizan mientras están conectadas a la toma de corriente, no habrá diferencia en el consumo de energía porque la corriente se reducirá para recolectar la misma cantidad de energía. En el caso de un caso de uso alimentado por batería, la diferencia será mucho mayor porque si la curva de descarga de la batería cae por debajo del voltaje de funcionamiento, el microprocesador se apagará.

Por lo tanto, las placas basadas en ESP tendrán un tiempo de ejecución más prolongado, ya que estas placas podrían funcionar a 4 V, mientras que a 4 V las placas Arduino se apagaron.

El Corazón del Sistema: El Reloj (Clock)

El reloj del sistema es el pulso que sincroniza todas las operaciones dentro del microcontrolador. Actúa como un metrónomo, generando una señal periódica de alta frecuencia que dicta el ritmo al que la CPU ejecuta instrucciones y los periféricos transfieren datos. La "inicialización del reloj" es uno de los primeros y más críticos pasos en la secuencia de arranque del hardware, ya que ningún otro componente puede funcionar de manera coordinada sin él. La frecuencia de este reloj es un parámetro de rendimiento clave.

La configuración del reloj del sistema tiene implicaciones directas tanto en el rendimiento como en la seguridad. Conceptualmente, una mayor frecuencia de reloj permite ejecutar más instrucciones por segundo, lo que se traduce en un mayor rendimiento. Sin embargo, esto tiene un coste: a mayor frecuencia, mayor es el consumo de energía, un factor crítico en dispositivos alimentados por batería.

Velocidad del reloj

Todas las placas Arduino funcionan a 16 MHz, lo que significa que el microprocesador puede ejecutar hasta 16 millones de instrucciones por segundo. Eso puede parecer mucho, pero si considera que simplemente configurar el pin digital en alto puede tomar más de 50 ciclos de reloj. Las tarjetas basadas en ESP son mucho más rápidas con una velocidad de reloj de 52 MHz a 240 MHz. La placa que se utilizará en la cátedra cuenta con un procesador *Tensilica Xtensa LX6 de doble núcleo de 32 bits de hasta 240 MHz*.

Memorias: FLASH, SRAM, ROM y EEPROM

La gestión de la memoria en sistemas embebidos es un factor determinante en la arquitectura del sistema, impactando directamente su rendimiento, coste y fiabilidad. A diferencia de los sistemas de cómputo de propósito general, los microcontroladores operan con recursos estrictamente limitados, donde cada byte cuenta. Una comprensión profunda de la arquitectura de memoria no es solo una cuestión de optimización; es un pilar esencial para garantizar el rendimiento, la estabilidad y, de manera crucial, la seguridad del firmware.

Los microcontroladores modernos, como el ESP32, utilizan una jerarquía de memoria especializada para distintas funciones. Cada tipo de memoria tiene un propósito específico, y su correcta utilización es clave para el diseño de un sistema eficiente.

Tipo de Memoria	Función y Características
Flash	Memoria no volátil (persiste sin energía) utilizada para almacenar el firmware principal (el programa del usuario) y, opcionalmente, datos de configuración o un bootloader secundario.
RAM	Memoria volátil (se pierde sin energía) de alta velocidad. Es el espacio de trabajo del microcontrolador donde se ejecuta el código y se almacenan las variables de tiempo de ejecución.
ROM	Memoria no volátil de solo lectura, grabada por el fabricante. Contiene el bootloader primario, un código inmutable que se ejecuta al arrancar para inicializar el hardware esencial.

La organización de estos recursos se define en el mapa de memoria, una estructura lógica que asigna direcciones físicas a los diferentes componentes del software. Este mapa es crucial, ya que dicta dónde residen el código ejecutable, las variables estáticas, los datos dinámicos y las estructuras de control del programa. Los segmentos más importantes de este mapa son:

- Segmento de código (o text): contiene las instrucciones ejecutables del programa, es decir, el firmware compilado. Normalmente reside en memoria no volátil (por ejemplo, Flash), lo que permite que el sistema conserve el programa aun cuando se pierde la alimentación. Este segmento suele ser de solo lectura durante la ejecución normal.
- Segmento de datos inicializados (data): almacena las variables globales y estáticas que tienen un valor inicial explícito definido en el código. Durante el arranque del sistema, estos valores son copiados desde la memoria no volátil hacia la memoria RAM, donde podrán ser leídos y modificados durante la ejecución.
- Segmento BSS (Block Started by Symbol): contiene las variables globales y estáticas que no fueron inicializadas explícitamente. Por convención, este segmento se inicializa en cero durante el proceso de arranque. Al igual que el segmento data, reside en RAM durante la ejecución.
- Stack (pila): es una región de memoria utilizada para el manejo de llamadas a funciones, almacenamiento de variables locales, direcciones de retorno y contexto de ejecución. Su crecimiento y decrecimiento es dinámico y depende del flujo del programa. Un uso inadecuado de la pila puede provocar desbordes (stack overflow), una fuente común de fallos y vulnerabilidades en sistemas embebidos.
- Heap: es la zona de memoria destinada a la asignación dinámica, utilizada por mecanismos como malloc y free en C/C++. Su tamaño y uso dependen de la aplicación. La fragmentación del heap y la falta de control sobre las asignaciones dinámicas pueden generar inestabilidad y problemas de seguridad, especialmente en sistemas con recursos limitados.

En conjunto, estos segmentos definen cómo se organiza y utiliza la memoria del sistema embebido. Comprender su funcionamiento es fundamental no solo para escribir firmware eficiente y confiable, sino también para identificar riesgos de seguridad asociados a errores de programación, corrupción de memoria y comportamientos no deseados del sistema.

Análisis Comparativo: Stack vs. Heap

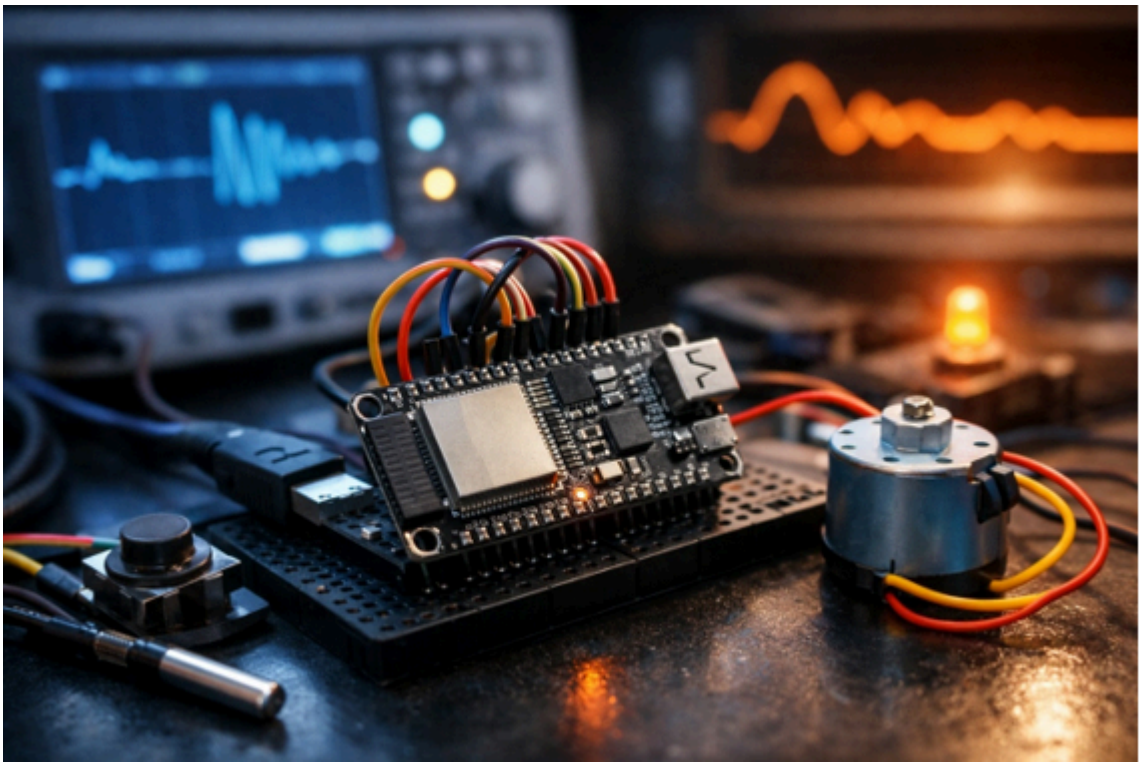
Aunque ambos residen en la RAM, el Stack y el Heap presentan funcionamientos, contenidos y perfiles de riesgo distintos, fundamentales para un arquitecto de sistemas.

Característica	Stack	Heap
Contexto típico	Direcciones de retorno de funciones, variables locales, registros salvados y contexto de interrupciones.	Datos asignados dinámicamente (malloc, new) cuyo tamaño o ciclo de vida no se conoce en tiempo de compilación.
Riesgos de seguridad asociados	**Stack Overflow:** Un desbordamiento de búfer puede sobrescribir direcciones de retorno, permitiendo la ejecución de código arbitrario.	**Heap Overflow:** Similar al de stack, pero corrompe datos adyacentes en el heap. Además, es susceptible a **Use-after-free**, **Double free** y **Corrupción de metadatos** del gestor de memoria.

La gestión meticulosa de la memoria es un vector crítico de seguridad en sistemas embebidos. La naturaleza de bajo nivel de la programación en C/C++, sumada a la ausencia de protecciones de memoria avanzadas como las Unidades de Gestión de Memoria (MMU) en muchos microcontroladores, expone a estos dispositivos a vulnerabilidades fácilmente explotables.

GPIO y Manejo de Señales

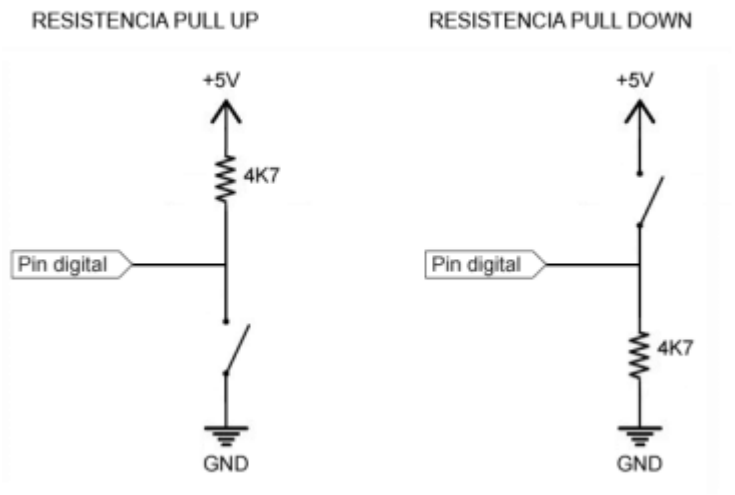
Los pines de Entrada/Salida de Propósito General (GPIO) son la interfaz fundamental entre el dominio digital del microcontrolador y el mundo físico, que es inherentemente analógico. Actúan como los "sentidos" y "actuadores" del sistema, permitiendo leer el estado de un pulsador, medir la temperatura con un sensor o encender un motor. Su correcta configuración y manejo son estratégicos para cualquier proyecto embebido.



Los microcontroladores poseen diferentes tipos de pines, cada uno capaz de cumplir con determinadas funciones. Entre los más importantes podemos destacar:

Entradas y Salidas Digitales

Es la forma más básica de interacción. Un pin configurado como salida digital puede establecer dos estados lógicos: HIGH (alto, típicamente 3.3V o 5V) o LOW (bajo, 0V), permitiendo controlar dispositivos simples como LEDs o relés. Como entrada digital, el pin puede leer si la señal externa conectada a él está en un nivel alto o bajo. Las entradas digitales deben llevar asociadas Resistencias de Pull-up y Pull-down. Cuando un pin de entrada no está conectado activamente a HIGH o LOW (por ejemplo, un pulsador no presionado), su estado es "flotante" e indeterminado, lo que puede provocar lecturas erróneas. Las resistencias de pull-up (conectan el pin a HIGH por defecto) y pull-down (conectan a LOW) aseguran que el pin siempre tenga un estado lógico definido, eliminando la ambigüedad.



Se pueden conectar:

- pulsadores
- sensores discretos
- llaves / interruptores

Así como las entradas digitales permiten sensar actividades que tienen dos estados, las salidas digitales permiten controlar actividades que tienen dos estados. Con las salidas digitales se puede encender o apagar el dispositivo que se conecte al pin.

Se pueden conectar:

- Leds
- Pantallas LCD
- Transistores
- Relés

Los pines de salida configurarse de la siguiente manera: const int led = 2;

```
void setup() {
    pinMode(led, OUTPUT);
}
void loop() {
    digitalWrite(led, HIGH);
    delay(500);
```

```
    digitalWrite(led, LOW);
    delay(500);
}
```

Mientras que los pines de entrada se pueden configurar de la siguiente manera:

```
const int boton = 4;
void setup() {
    pinMode(boton, INPUT_PULLUP);
    Serial.begin(115200);
}
void loop() {
    int estado = digitalRead(boton);
    if (estado == LOW) {
        Serial.println("Botón presionado");
    }
}
```

Entradas Analógicas

El mundo físico raramente se comporta de manera binaria. Magnitudes como la intensidad lumínica, la temperatura, la presión o la posición de un potenciómetro varían de forma continua en el tiempo y en amplitud. Sin embargo, los microcontroladores operan exclusivamente sobre información digital, es decir, valores discretos representados en forma binaria. El nexo entre ambos dominios es el Conversor Analógico–Digital (ADC, Analog-to-Digital Converter). El ADC es un periférico que toma una señal de tensión analógica —generalmente comprendida entre 0 V y un voltaje de referencia V_REF, que suele coincidir con el voltaje de alimentación del microcontrolador— y la transforma en un número entero que puede ser procesado por la CPU. La característica fundamental de un ADC es su resolución, expresada en bits. Un ADC de Nbits puede representar: 2^N " valores discretos"

Por ejemplo: 8 bits → 2^8=256valores (0 a 255) 10 bits → 2^10=1024valores (0 a 1023) 12 bits → 2^12=4096valores (0 a 4095)

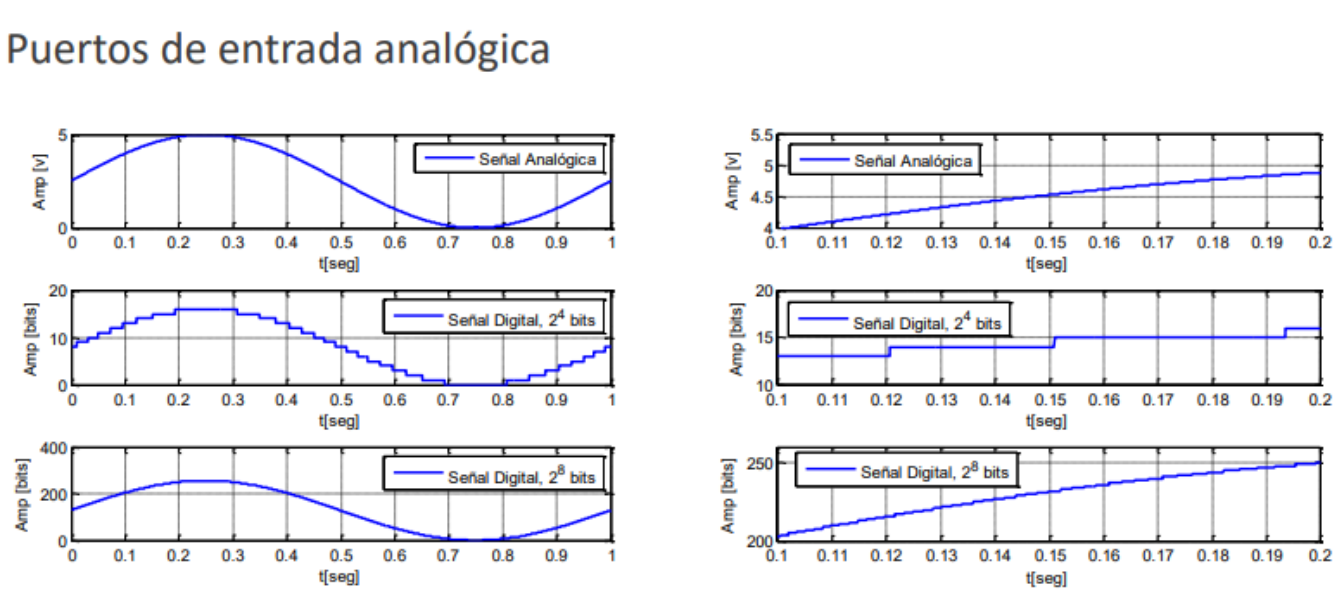
Cada uno de estos valores representa un intervalo de tensión del rango total de medición. A este intervalo se lo denomina resolución en tensión o LSB (Least Significant Bit), y se calcula como:

> **Resolución** ≈ V_REF/2^N

Por ejemplo, con un ADC de 12 bits y un V_REF=3.3" V" :

```
**Resolución** ≈3.3/4096≈0.8 mV
```

Esto significa que dos tensiones que difieran en menos de 0.8 mV serán indistinguibles para el sistema digital.



Los pines ADC pueden configurarse de la siguiente manera:

```
// Definición del pin analógico
const int adcPin = 34;   // GPIO 34 (solo entrada en ESP32)
void setup()
{
    // Configuración del ADC
    analogReadResolution(12);     // ADC de 12 bits → valores de 0 a 4095
}

void loop()
{
    int adcValue = analogRead(adcPin);   // Lee el ADC (0–4095)
}
```

Modulación por Ancho de Pulso (PWM)

Los microcontroladores generan salidas digitales, es decir, señales que solo pueden tomar dos estados: nivel bajo (LOW) o nivel alto (HIGH). Sin embargo, muchas aplicaciones requieren controlar magnitudes que parecen analógicas, como el brillo de un LED, la velocidad de un motor de CC o la posición de un servo. Para lograrlo, se emplea la técnica de Modulación por Ancho de Pulso (PWM, Pulse Width Modulation).

En PWM no se varía el valor del voltaje de salida, sino el tiempo relativo durante el cual la señal permanece en estado HIGH dentro de un período fijo.

> Duty cycle = t / T
>
> t = tiempo en parte positiva
>
> T = Periodo, tiempo total

A este porcentaje se lo denomina ciclo de trabajo (duty cycle) y se expresa típicamente como un valor entre 0 % y 100 %. 0 % → la salida permanece siempre en LOW 50 % → la salida está en HIGH la mitad del tiempo 100 % → la salida permanece siempre en HIGH

La señal PWM es una onda cuadrada de frecuencia constante, pero con ancho de pulso variable. El efecto de esta señal sobre el sistema controlado depende de su naturaleza física:

- En un LED, el ojo humano integra la energía luminosa y percibe un brillo proporcional al ciclo de trabajo.
- En un motor, la inercia mecánica y eléctrica actúa como un filtro pasa bajos, resultando en una velocidad promedio proporcional al duty cycle.

Al igual que en el ADC, el PWM posee una resolución, determinada por la cantidad de bits usados para representar el ciclo de trabajo. Si el PWM tiene una resolución de Nbits, el duty cycle puede tomar:

```
        2^N valores discretos
```

Por ejemplo:

- 8 bits → 256 niveles (0 a 255)
- 10 bits → 1024 niveles (0 a 1023)
- 12 bits → 4096 niveles (0 a 4095)

Cada incremento representa un pequeño cambio en el ancho del pulso, y por lo tanto, en la energía promedio entregada a la carga.

La modulación PWM se puede utilizar de la siguiente manera:

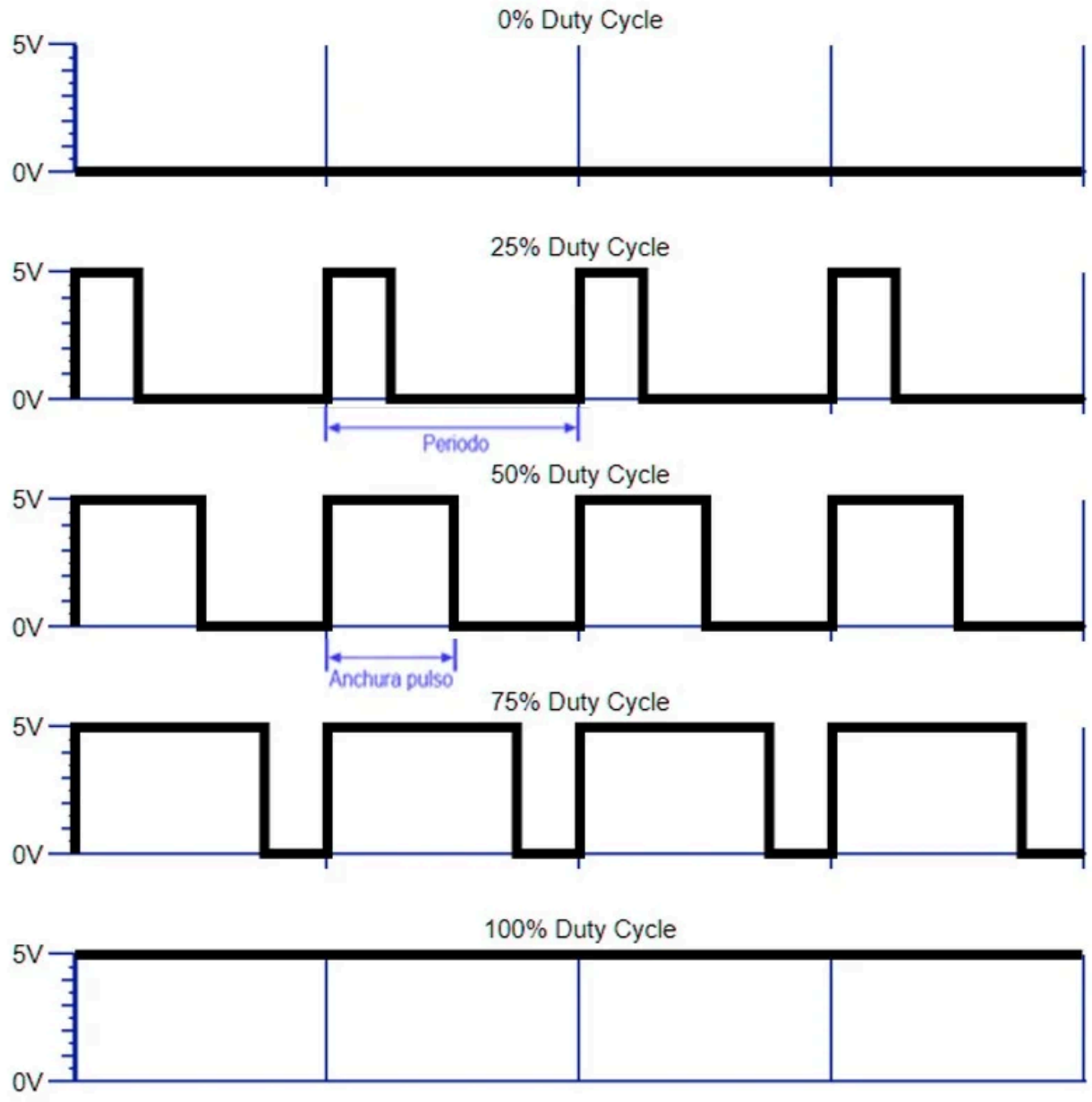
```
const int ledPin = 25;   // Pin con salida PWM

void setup()
{
    pinMode(ledPin, OUTPUT);
}

void loop()
{
    // Encender el LED gradualmente
    for (int value = 0; value <= 255; value++)
    {
        analogWrite(ledPin, value);  // 0–255 → 0–100% duty cycle
        delay(10);
    }

    // Apagar el LED gradualmente
    for (int value = 255; value >= 0; value--)
    {
        analogWrite(ledPin, value);
        delay(10);
    }
}
```

La siguiente imagen muestra diferentes ciclos de trabajos para un rango de voltajes de 0 a 5v.



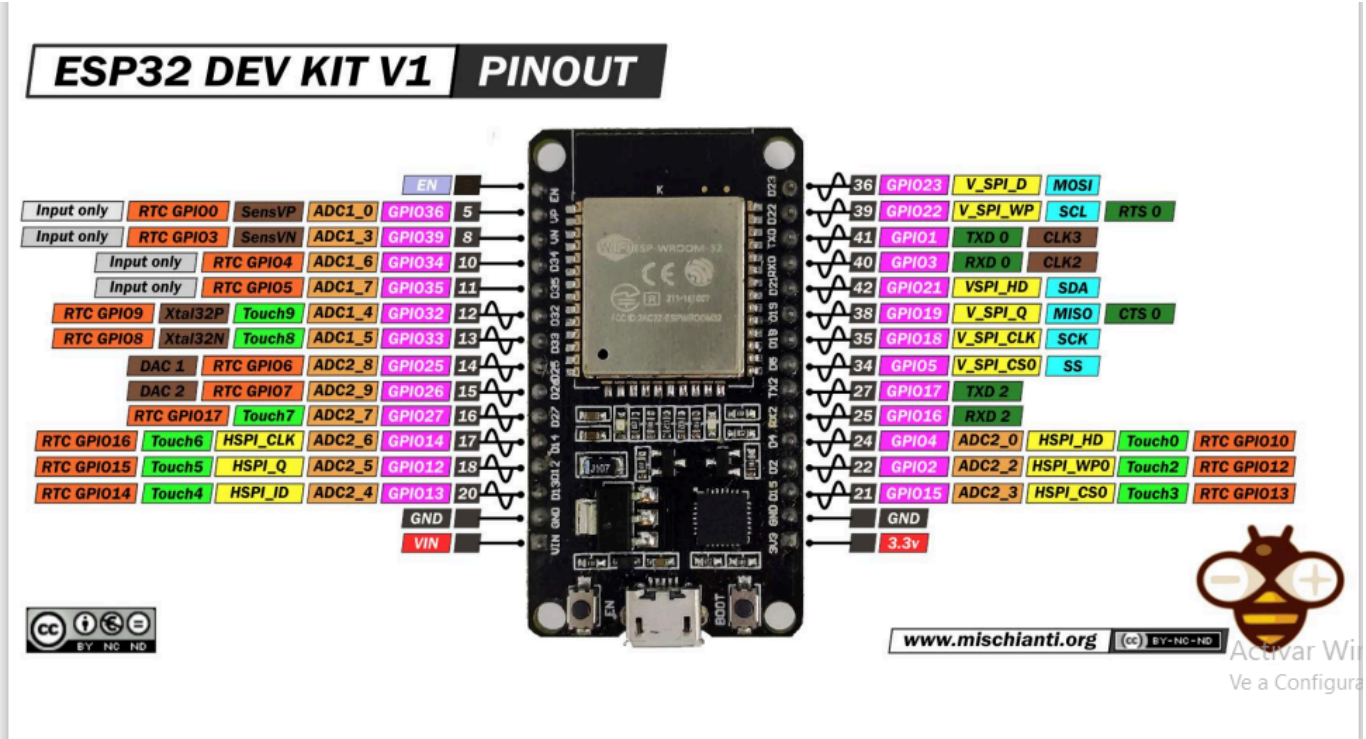
Por ejemplo, si con una tensión Vcc de 5V queremos una señal PWM de 1V, se generará una señal que el 20% del tiempo valdrá 5V y el 80% restante 0V.

Esta modulación es muy usada para controlar la cantidad de energía que se envía a una carga, es una técnica utilizada para regular la velocidad de giro de los motores, regulación de intensidad luminosa, controles de elementos termoeléctricos o controlar fuentes conmutadas entre otros usos.

Para comprender más sobre el uso de esta técnica se deja el siguiente video.

Vale mencionar que la placa cuenta solo con dos pines que pueden utilizarse como salidas analógicas puras: **GPIO25 y GPIO26** mediante un Conversor Digital Analógico (DAC) con una resolución de 8 bits (con valores de 0 a 255).

La siguiente imagen sintetiza la distribución de conectores de la placa.



- Conector micro-USB para la alimentación y comunicación
- 30 ó 36 Conectores GPIO para entrada y salida de señales digitales
- 3 interfaces serie SPI (HW específico)
- 2 interfaces I2C (HW específico)
- 2 interfaces I2S
- 2 canales de conversor analógico a digital de 12 bits (ADC)
- 2 canales de conversor digital a analógico (DAC)
- 16 canales de salida PWM
- 3 interfaces UART
- 10 GPIO de detección capacitiva (Touch)
- Botones ENABLE y BOOT
- Conectividad Wi-Fi y Bluetooth
- LED integrado

Consideraciones Eléctricas para la Integridad del Hardware

La interacción con componentes externos introduce riesgos eléctricos que pueden dañar permanentemente el microcontrolador si no se manejan con cuidado.

- Niveles Lógicos y Tensión Máxima: Cada pin tiene un rango de voltaje máximo que puede tolerar. Conectar una señal de 5V a un pin de un microcontrolador de 3.3V sin un divisor de tensión o un convertidor de nivel lógico puede causar daños irreparables.
- Corriente Máxima: Los pines GPIO solo pueden suministrar o drenar una cantidad limitada de corriente. Exceder este límite, por ejemplo, al conectar directamente un motor, puede sobrecalentar y destruir el pin o el microcontrolador.
- Acondicionamiento de Señales: Las señales del mundo real a menudo son "ruidosas" o no son eléctricamente compatibles con los pines del MCU. El acondicionamiento de señales es necesario para garantizar lecturas correctas y proteger el hardware. Las técnicas comunes incluyen: Divisores resistivos para adaptar niveles de voltaje, Filtros RC para eliminar ruido, Amplificadores operacionales para

amplificar señales débiles, Convertidores de nivel lógico (level shifters) para una interfaz segura entre lógicas de diferente voltaje, y protección con diodos para evitar sobretensiones.

Protocolos de comunicación

Los puertos de comunicación permiten que un sistema embebido intercambie información con otros dispositivos, sensores, actuadores o sistemas de mayor nivel, como computadoras y redes. A diferencia de las entradas y salidas simples, estos puertos implementan protocolos de comunicación, es decir, reglas precisas que definen cómo se envían los datos, en qué formato, a qué velocidad y con qué mecanismos de sincronización y control de errores. Gracias a ellos, los microcontroladores pueden integrarse en sistemas más complejos, cooperar con otros nodos y formar parte de arquitecturas distribuidas.

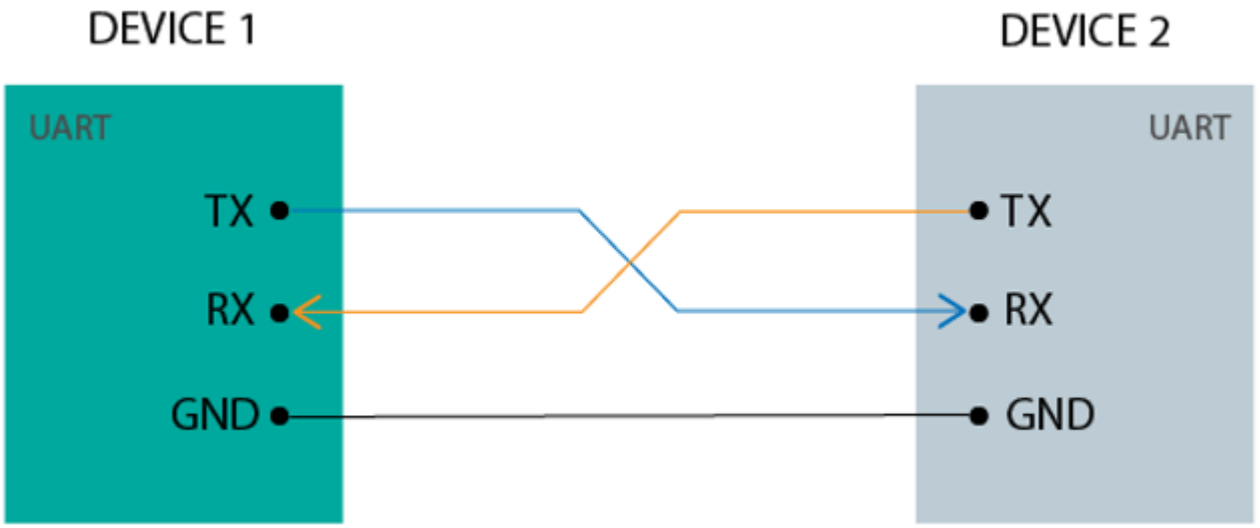
Para que puedan comunicarse, dos dispositivos tienen que acordar ciertas cosas, entre ellas la velocidad a la cual los datos serán enviados y leídos, y los niveles de voltaje representando a un 1 y a un 0. A continuación presentaremos algunos protocolos de comunicación.

La comunicación entre dispositivos puede ser asíncrona o síncrona, dependiendo de cómo se sincronizan los datos entre el emisor y el receptor. En la comunicación asíncrona, no se requiere una señal de reloj compartida; en su lugar, los datos se envían junto con bits de inicio y de parada que permiten al receptor identificar el comienzo y el fin de cada paquete de información. Este enfoque simplifica el cableado y la interconexión de dispositivos, siendo ideal para enlaces punto a punto, aunque generalmente es más lento y menos eficiente que la comunicación síncrona. Por otro lado, en la comunicación síncrona, ambos dispositivos comparten una misma señal de reloj o sincronización, lo que permite que los datos se transmitan de manera continua y coordinada, sin necesidad de indicadores de inicio o fin de cada byte. Este tipo de comunicación es eficiente para altas velocidades y para sistemas donde la precisión temporal es crítica, como en interfaces SPI o I^2C.

Comunicación Asíncrona: UART

La comunicación serie asíncrona, implementada a través del protocolo UART (Universal Asynchronous Receiver-Transmitter), es una de las herramientas más importantes en sistemas embebidos. Su utilidad principal incluye la depuración de código, permitiendo mostrar mensajes en una consola, y la interconexión punto a punto con otros dispositivos. Su simplicidad y bajo requerimiento de pines lo hacen ideal para conectar módulos externos, como receptores GPS o módulos de sensores, que transmiten datos de manera continua.

El protocolo UART utiliza una interfaz de comunicación serie basada en dos líneas principales: TX (transmisión) y RX (recepción). El pin TX de un dispositivo se conecta al pin RX del otro, y viceversa, estableciendo un canal de comunicación bidireccional simple. Para que los dispositivos interpreten correctamente los datos, ambos deben estar configurados con el mismo baudrate, es decir, la velocidad de transmisión medida en bits por segundo (bps). Por ejemplo, un baudrate de 115200 bps indica que se envían 115200 bits cada segundo.



En el ecosistema ESP32, es común utilizar un conversor de USB a UART para conectar el microcontrolador a un ordenador, lo que facilita tanto la programación como la depuración de aplicaciones. Aunque UART es muy útil, su naturaleza punto a punto lo hace poco adecuado para sistemas que requieren comunicación con múltiples dispositivos en una misma línea, donde protocolos de bus como I^2C, SPI o CAN resultan más apropiados.

Ejemplo de configuración de UART en ESP32 usando Arduino:

```
// Configuración de UART
void setup() {
  Serial.begin(115200); // Inicia UART a 115200 bps
  Serial.println("UART inicializada");
}

void loop() {
  // Enviar un mensaje al ordenador
  Serial.println("Hola desde ESP32");

  // Leer datos si están disponibles
  if (Serial.available()) {
    char c = Serial.read();
    Serial.print("Recibido: ");
    Serial.println(c);
  }

  delay(500);
}
```

Ejercitación

> **Caso práctico 01**: desarrollar un sketch que permita encender/apagar un juego de LEDs (uno azul y otro verde) utilizando para ello el puerto serie. La idea es utilizar los dos leds que ya vienen provistos en la placa UTN (uno verde más el led propio de la Esp32) y procesar las siguientes entradas por puerto serie:

> - **v** o **V** para encender el LED verde y apagar el azul
> - **a** o **A** para encender el LED azul y apagar el verde

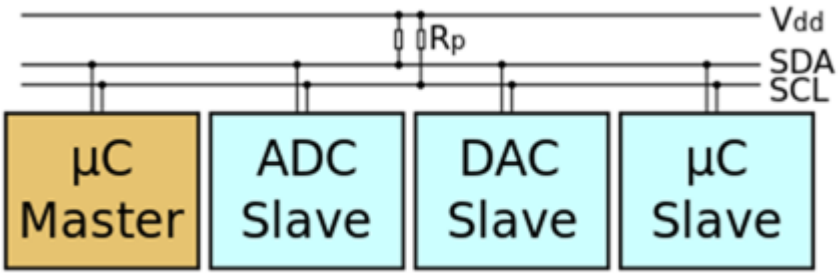
Para ver el código de la solución acceder a esta carpeta: Práctico 01

> **Caso práctico 02**: desarrollar un sketch que permita >procesar un comando (en formato texto) recibido por el puerto >serie tal que: "LUZ_ON": encienda el LED y active un relay (que podría estar > conectado con el sistema de ilumninación de una habitación). "LUZ_OFF": con apague el LED y desactive el relay.

Para ver el código de la solución acceder a esta carpeta: Práctico 02

Comunicación Síncrona I2C

El protocolo I2C es una interfaz de bus que se caracteriza por su arquitectura de solo dos hilos: SDA (Serial Data) para la línea de datos y SCL (Serial Clock) para la señal de reloj. Su diseño maestro-esclavo incluye un sistema de direccionamiento único de 7 bits para cada dispositivo, lo que permite que múltiples periféricos esclavos coexistan en el mismo bus, todos controlados por un único dispositivo maestro, como el ESP32.



El uso de I2C es prevalente en proyectos con ESP32 por su eficiencia en el uso de pines.

- Sensores: Es ideal para conectar una multitud de sensores en un mismo bus, como los de presión barométrica (BMP180), luz ambiental (APDS-9930) o campos magnéticos (HMC5883L).
- Expansores de Puertos: Cuando se agotan los pines GPIO del ESP32, se pueden añadir más utilizando expansores como el PCF8574 (8 bits) o el PCF8575 (16 bits).
- Relojes de Tiempo Real (RTC): Para mantener la hora exacta incluso cuando el ESP32 está apagado, se utilizan módulos RTC como el DS1307, que se comunican vía I2C.

En el entorno de desarrollo de Arduino para ESP32, la interacción con el bus I2C se gestiona a través de la librería Wire. Esta librería abstrae la complejidad del protocolo, permitiendo al desarrollador iniciar la comunicación y leer los datos del sensor con unas pocas líneas de código.

Ejemplo de inicialización y uso de un sensor de temperatura BMP280.

#include <Wire.h> // Librería para comunicación I2C #include <Adafruit_Sensor.h> #include <Adafruit_BME280.h> // Librería para el sensor BME280

```
#define I2C_ADDRESS 0x76     // Dirección I2C del sensor

Adafruit_BME280 sensor;     // Objeto sensor
```

```
void setup() {
  Serial.begin(115200);

  // Inicializar bus I2C
  Wire.begin();

  // Inicializar sensor
  if (!sensor.begin(I2C_ADDRESS)) {
    Serial.println("No se pudo encontrar el sensor BME280. Verificar
conexiones.");
    while (1); // Detener ejecución
  }

  Serial.println("Sensor I2C inicializado correctamente");
}

void loop() {
  float temperatura = sensor.readTemperature();
  float humedad = sensor.readHumidity();
  float presion = sensor.readPressure() / 100.0F; // hPa

  Serial.print("Temp: "); Serial.print(temperatura); Serial.print(" °C, ");
  Serial.print("Humedad: "); Serial.print(humedad); Serial.print(" %, ");
  Serial.print("Presión: "); Serial.print(presion); Serial.println(" hPa");

  delay(1000);
}
```

I2C se utiliza con frencuencia par conectar periféricos de menor velocidad a la placa. Uno de estos periféricos suelen ser las pantallas o módulos LCD. La placa de desarrollo UTN viene integrada con una pantalla OLED para arduino con las conexiones de pines ya configuradas.

Para conocer qué es y comó funciona en detalle este protocolo, tal como se muestra en el video de este link: I2C explicado.

> **Caso práctico 03**: refactorizar el sketch del práctico anterior para mostrar en el módulo OLED el estado de los componentes gestionados por el puerto serie.
>
> **Nota**: para poder utilizar este módulo es necesario importar la librería **Adafriut SH110X**.

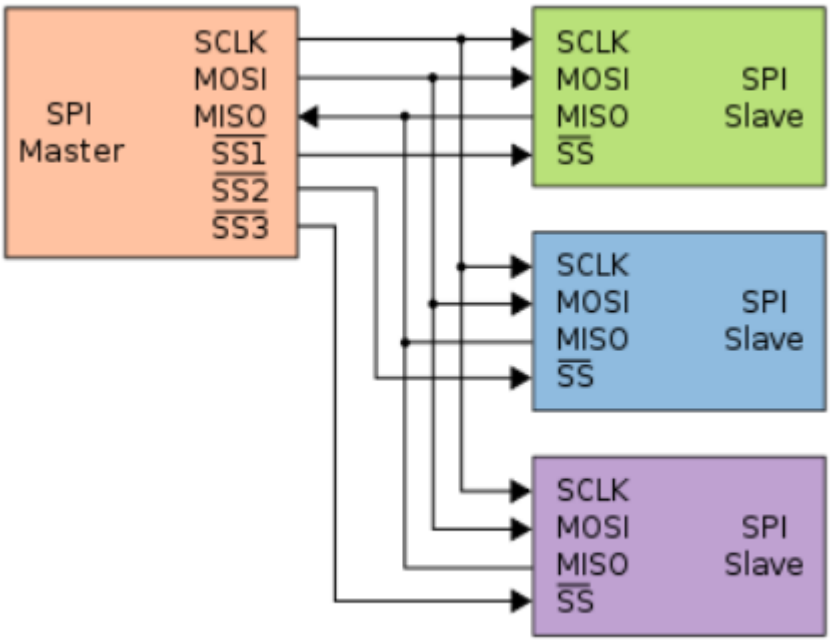
Para ver el código de la solución acceder a esta carpeta: Práctico 03

Comunicación Síncrona SPI

El protocolo SPI es una interfaz de comunicación síncrona maestro-esclavo diseñada para transferencias de datos a alta velocidad. A diferencia de I2C, SPI utiliza típicamente cuatro líneas, lo que le permite alcanzar un mayor rendimiento:

- MOSI (Master Out Slave In): Línea de datos del maestro al esclavo.
- MISO (Master In Slave Out): Línea de datos del esclavo al maestro.
- SCLK (Serial Clock): Señal de reloj generada por el maestro.

- CS (Chip Select): Línea utilizada por el maestro para seleccionar el esclavo específico con el que desea comunicarse.



Su alta velocidad hace que SPI sea la opción preferida para periféricos que requieren un gran ancho de banda, tales como pantallas gráficas o tarjetas de memoria.

Aquí tienes un ejemplo de uso.

```
#include <SPI.h>
#include <SD.h>

#define SD_CS 5   // Pin Chip Select de la tarjeta SD

void setup() {
  Serial.begin(115200);

  // Inicializar SD
  if (!SD.begin(SD_CS)) {
    Serial.println("Error al inicializar la tarjeta SD");
    while (1); // Detener ejecución si falla
  }
  Serial.println("Tarjeta SD inicializada correctamente");

  // Crear o abrir archivo para escribir
  File archivo = SD.open("/registro.txt", FILE_WRITE);
  if (archivo) {
    archivo.println("Hola desde ESP32");
    archivo.close(); // Siempre cerrar el archivo
    Serial.println("Se escribió en el archivo correctamente");
  } else {
    Serial.println("No se pudo abrir el archivo");
  }
```

```
}

void loop() {
  // Leer contenido del archivo
  File archivo = SD.open("/registro.txt");
  if (archivo) {
    Serial.println("Contenido del archivo:");
    while (archivo.available()) {
      Serial.write(archivo.read());
    }
    archivo.close();
  } else {
    Serial.println("No se pudo abrir el archivo");
  }

  delay(5000); // Espera antes de volver a leer
}
```

Comunicación en Red sobre TCP/IP

El stack de protocolos TCP/IP es la columna vertebral de la conectividad en redes modernas, tanto cableadas (Ethernet) como inalámbricas (WiFi). Gracias a la integración de la pila de red ligera lwip (Lightweight IP), el ESP32 puede operar como un nodo de primera clase en redes IP. Esto abre la puerta a un sinfín de aplicaciones de IoT, permitiendo que el dispositivo actúe como un servidor web, un cliente de APIs REST, o se comunique mediante protocolos en tiempo real como WebSockets.

Protocolos Fundamentales: TCP y UDP

En la capa de transporte de TCP/IP, dos protocolos dominan la escena, cada uno con implicaciones críticas para sistemas embebidos:

- TCP (Transmission Control Protocol): Es un protocolo orientado a la conexión que garantiza la entrega ordenada y sin errores de los datos. Sin embargo, esta fiabilidad impone una sobrecarga significativa de memoria y CPU, un factor crucial en un dispositivo con recursos limitados como el ESP32. La necesidad de mantener el estado de la conexión, gestionar búferes de transmisión y recepción, y ejecutar la lógica de retransmisión consume recursos valiosos.
- UDP (User Datagram Protocol): Es un protocolo sin conexión, mucho más simple y rápido. Su naturaleza "dispara y olvida" (fire-and-forget) lo hace ideal para aplicaciones de IoT que envían datos de sensores con alta frecuencia y donde la pérdida ocasional de un paquete es aceptable. Su bajo consumo de recursos lo convierte en la opción preferida para telemetría en tiempo real, donde la velocidad y la eficiencia energética son más importantes que la fiabilidad absoluta de cada paquete.

Protocolos de Aplicación: HTTP y WebSockets

Sobre la base de TCP/IP se construyen los protocolos de aplicación que dan forma a los servicios de red.

- HTTP/REST: El protocolo REST (Representational State Transfer) es un estilo de arquitectura de software construido sobre HTTP. Permite que el ESP32 interactúe con servicios web externos (APIs) como cliente (por ejemplo, para obtener datos meteorológicos) o que actúe como un proveedor de servicios

(servidor REST), exponiendo sus propias funcionalidades, como la lectura de un sensor, a través de una API.
- WebSockets: Esta tecnología establece una única conexión TCP persistente entre un cliente (como un navegador web) y un servidor (el ESP32). A través de esta conexión, ambas partes pueden enviar datos en cualquier momento, lo que permite una comunicación bidireccional y en tiempo real, perfecta para paneles de control interactivos o sistemas de notificaciones instantáneas.

Comunicación Inalámbrica de Corto Alcance: Bluetooth

Una de las características más potentes y distintivas del ESP32 es su capacidad de comunicación Bluetooth dual, soportando tanto Bluetooth Clásico (BR/EDR) como Bluetooth Low Energy (BLE). Esta dualidad le otorga una flexibilidad excepcional, permitiéndole conectar periféricos que requieren un flujo de datos continuo y alto ancho de banda, como dispositivos de audio o transferencia de datos, así como sensores IoT de bajo consumo que pueden operar durante meses o años con una sola batería.

Bluetooth Clásico (BR/EDR)

Bluetooth Clásico está diseñado para aplicaciones que demandan un flujo de datos constante y mayor ancho de banda que BLE.

- Identificación: En sistemas operativos como Linux, herramientas como hciconfig lo identifican como un dispositivo de tipo BR/EDR (Basic Rate/Enhanced Data Rate).
- Arquitectura: La red básica se denomina piconet, compuesta por un dispositivo maestro que puede conectarse a uno o varios dispositivos esclavos.
- Direccionamiento: Cada dispositivo Bluetooth posee una dirección física única de 48 bits, conocida como BD_ADDR (Bluetooth Device Address), que lo identifica de manera inequívoca dentro de la red.

Bluetooth Low Energy (BLE)

BLE fue diseñado desde el inicio para un consumo de energía ultra bajo, lo que lo hace ideal para dispositivos IoT que deben funcionar durante largos períodos con baterías pequeñas.

- Roles: Los dispositivos BLE asumen principalmente dos roles: Central (inicia la conexión, como un smartphone) y Periférico (ofrece el servicio, como un sensor de temperatura). El dispositivo central escanea el entorno y el periférico anuncia su presencia.
- Comunicación: La comunicación se basa en un paradigma de anuncios (advertising) y escaneo (scanning). Los periféricos emiten periódicamente paquetes de anuncio con información sobre sí mismos, y los dispositivos centrales los detectan para descubrir y, opcionalmente, conectarse con ellos.
- Estructura de Datos (GATT): Una vez establecida la conexión, el intercambio de datos se organiza mediante el Perfil de Atributos Genérico (GATT). GATT estructura los datos jerárquicamente en servicios y características. Un servicio agrupa datos relacionados (por ejemplo, "Servicio de Frecuencia Cardíaca"), y una característica es un valor de dato específico dentro de ese servicio (por ejemplo, "Medición de Frecuencia Cardíaca"). En esta arquitectura, el periférico actúa como servidor GATT, alojando la base de datos de atributos, mientras que el dispositivo central actúa como cliente GATT, descubriendo y consumiendo esos datos.

Tabla Comparativa de Protocolos

La elección de un protocolo de comunicación es una decisión de diseño crítica que impacta directamente en el consumo de energía, la velocidad de transferencia, el alcance físico y la complejidad del software. No existe una solución única para todos los problemas; la mejor elección depende de los requisitos específicos del proyecto. La siguiente tabla resume las características clave de cada protocolo discutido en esta guía para ayudar a tomar una decisión informada.

Protocolo	Tipo de Interfaz	Ventajas Clave	Limitaciones Principales	Caso de Uso Típico
UART	Serie, Asíncrono, Punto a Punto	Simplicidad, bajo número de pines (2), ideal para depuración.	Baja velocidad, comunicación solo entre dos dispositivos.	Conectar módulos GPS, depuración por consola.
I2C	Serie, Síncrono, Bus Multidispositivo	Mínimo uso de pines (2) para múltiples dispositivos, direccionable.	Velocidad moderada, complejidad en el manejo del bus.	Lectura de datos de múltiples sensores. Control de pantallas
SPI	Serie, Síncrono, Bus Multidispositivo	Alta velocidad de transferencia, comunicación full-duplex.	Requiere un mínimo de 4 líneas de bus (MOSI, MISO, SCLK) más un pin de Chip Select (CS) por cada dispositivo esclavo, resultando en un total de 3+N pines para N esclavos.	Control de pantallas. Control de dispositivos de memoria.
TCP/IP	Red, Paquetes	Comunicación fiable y estandarizada, base para protocolos de alto nivel.	Alta sobrecarga de protocolo y consumo de memoria/CPU.	Crear un servidor web, conectarse a una API REST en la nube.
Bluetooth	Inalámbrico, Corto Alcance	Mayor tasa de transferencia que BLE, bien establecido para audio y datos.	Mayor consumo de energía que BLE.	Streaming de datos o aplicaciones que requieran una conexión persistente.

Actuadores y Sensores

El puente entre el mundo digital y el físico

Los sistemas embebidos constituyen la tecnología fundamental que permite a los dispositivos digitales percibir y actuar sobre el mundo físico. Son el puente invisible entre el procesamiento de datos y la interacción tangible, una capacidad que ha impulsado la era del Internet de las Cosas (IoT). La importancia

estratégica de comprender la interacción entre sensores, controladores y actuadores es, por tanto, primordial. Esta relación simbiótica es el núcleo de cualquier dispositivo "inteligente", desde un simple termostato conectado hasta complejos sistemas de automatización industrial.

El propósito de este bloque es proporcionar un análisis estructurado de estos componentes clave: los sensores que actúan como los sentidos del sistema, los actuadores que ejecutan sus acciones y los microcontroladores que sirven como su cerebro. Se explorará su interrelación, los desafíos prácticos de su implementación y los riesgos de ciberseguridad inherentes a su conectividad. Utilizando el versátil microcontrolador ESP32 como ejemplo práctico y los estándares de ciberseguridad emergentes como marco de referencia, esta guía busca ofrecer una visión integral del ciclo completo de percepción, procesamiento y acción que define a los sistemas físicos modernos. El primer componente en este ciclo, y nuestro punto de partida, es aquel que dota al sistema de la capacidad de percibir su entorno: el sensor.

Los Sentidos del Sistema: Una Taxonomía de Sensores

Los sensores son los "sentidos" de un sistema embebido, componentes transductores encargados de capturar datos del entorno y convertirlos en señales eléctricas que el microcontrolador puede procesar. La selección del sensor adecuado es un paso crítico en el diseño de cualquier sistema, ya que define qué puede percibir y con qué precisión. Estos dispositivos pueden ser tan variados como los sentidos humanos, midiendo luz, sonido, temperatura, movimiento y muchas otras variables físicas.

Una diferencia fundamental radica en la naturaleza de su señal de salida. Los sensores analógicos producen una señal de voltaje continua que es proporcional a la magnitud física que miden. Para que un microcontrolador digital pueda interpretar esta señal, necesita un convertidor analógico-digital (ADC). La placa de desarrollo ESP32, por ejemplo, integra múltiples canales de ADC (referenciados en su pinout como ADC0, ADC1, etc.), lo que le permite interactuar directamente con esta categoría de sensores. Por otro lado, los sensores digitales emiten señales discretas (unos y ceros) a través de protocolos de comunicación estandarizados como I2C o SPI, ofreciendo una salida ya procesada y a menudo más robusta frente al ruido eléctrico.

A continuación, se presenta una taxonomía de sensores comunes, catalogados según su función y basados en los modelos descritos en la documentación de desarrollo para la ESP32.

Categoría	Sensor	Función y Aplicación
Posicionamiento	GY-GPS6MV2	Módulo GPS basado en el receptor u-blox NEO-6M. Permite determinar la ubicación geográfica del dispositivo mediante señales GNSS, con una precisión típica del orden de algunos metros en condiciones normales. Se utiliza en aplicaciones de seguimiento, navegación y geolocalización.
Ambiental	BMP180	Sensor de presión barométrica que permite estimar la altitud y detectar variaciones atmosféricas. Se emplea en estaciones meteorológicas, dispositivos portátiles y sistemas de monitoreo ambiental.
Luz ambiental	BH1750FVI	Sensor digital de luz ambiental que mide la iluminancia del entorno. Se utiliza para ajustar automáticamente el brillo de pantallas o controlar sistemas de iluminación en función de las condiciones de luz.
Luz y proximidad	APDS-9930	Sensor óptico multipropósito que mide la luz ambiental y detecta la proximidad de objetos sin contacto físico. Es utilizado en interfaces de usuario, control automático de pantallas y detección de presencia.
Movimiento e inercia	MPU-6050	Sensor inercial que integra un acelerómetro y un giroscopio, permitiendo medir aceleración lineal y velocidad angular. Se utiliza para estimar orientación, detectar movimiento, inclinación o vibraciones.
Orientación	HMC5883L	Sensor magnetométrico que mide el campo magnético terrestre, permitiendo implementar una brújula digital para determinar la orientación cardinal.
Distancia	HC-SR04	Sensor de distancia que calcula la separación a un objeto midiendo el tiempo de retorno de un pulso ultrasónico. Se utiliza en robótica y sistemas de detección y evasión de obstáculos.
Tiempo	DS1307	Es un reloj de tiempo real (RTC) que mantiene la hora y la fecha de forma precisa, incluso si el microcontrolador se apaga

Una vez que el sistema ha percibido su entorno a través de estos sensores, el siguiente paso lógico es actuar en consecuencia, una tarea que recae sobre los actuadores.

La Acción en el Mundo Físico: Actuadores

Los actuadores son los componentes que traducen las señales digitales del controlador en acciones físicas, como movimiento, luz, sonido o calor. Son los "músculos" del sistema embebido, responsables de efectuar cambios en el entorno y completar así el ciclo de interacción. Si los sensores permiten que un sistema observe, los actuadores le permiten hacer. Esta capacidad de acción es lo que transforma a un dispositivo de un mero colector de datos en un agente activo en el mundo físico.

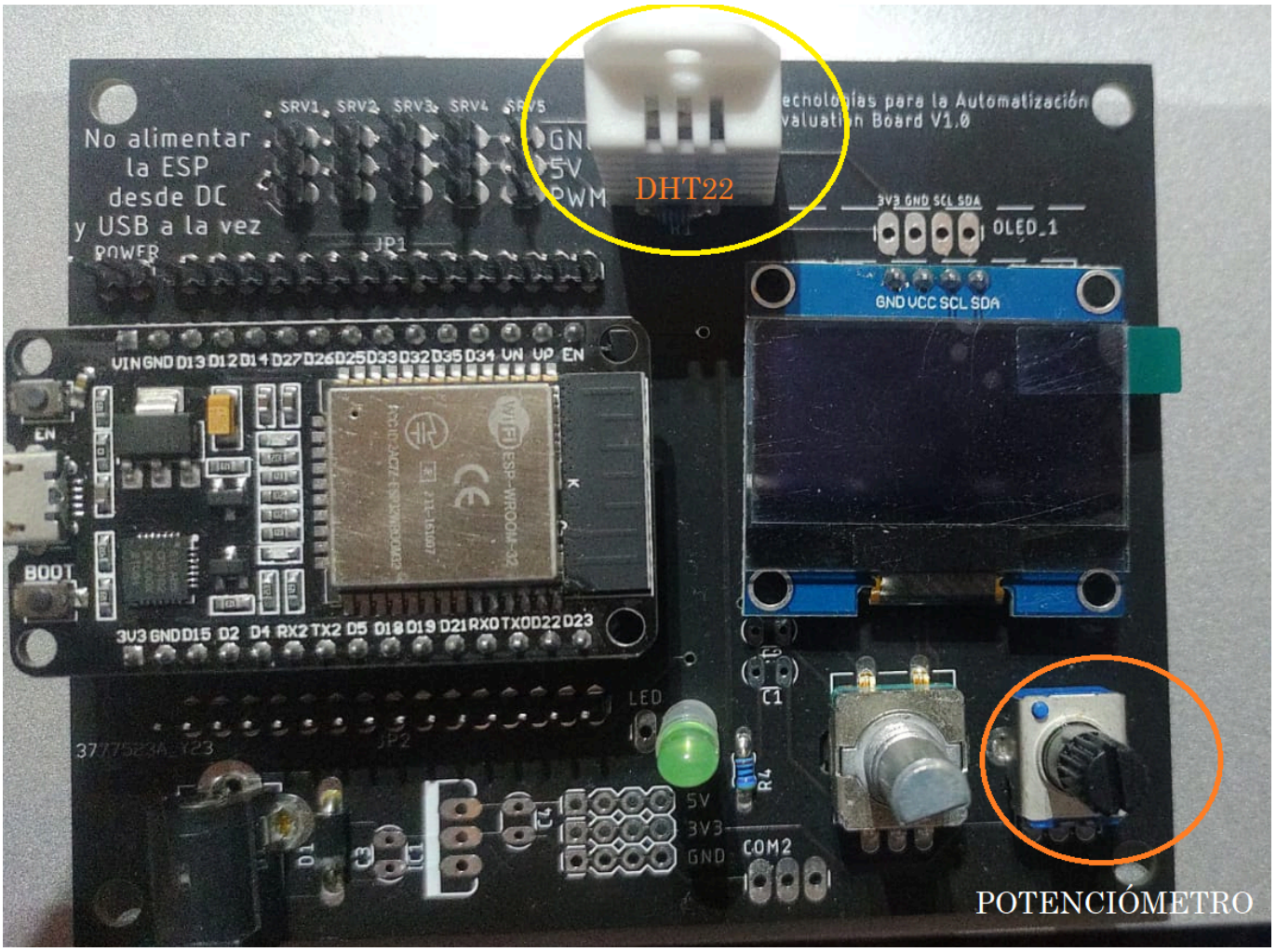
A continuación, se detallan dos tipos de actuadores comúnmente utilizados en proyectos con la ESP32:

- Servomotores: Un servomotor es un dispositivo de movimiento de precisión. A diferencia de un motor convencional que gira continuamente, el eje de un servo puede rotar a una posición angular específica, generalmente en un rango de 0 a 180 grados, y mantenerla. La posición se determina por la naturaleza de la señal de control que recibe del microcontrolador. Esta capacidad de posicionamiento controlado los hace ideales para aplicaciones como brazos robóticos, sistemas de dirección en vehículos a control remoto o cualquier mecanismo que requiera un movimiento preciso y repetible.
- NeoPixels (WS2812): Los NeoPixels son LEDs RGB direccionables individualmente que se controlan a través de una única línea de datos. Un flujo de 24 bits (8 bits para rojo, 8 para verde y 8 para azul) se envía en serie para establecer el color y el brillo de cada LED en una cadena. La principal particularidad de estos dispositivos es que requieren una temporización de señal extremadamente precisa, con pulsos del orden de los 400 nanosegundos (ns). La ESP32 cuenta con un periférico de hardware especializado llamado
- Transceptor de Control Remoto (RMT): diseñado para generar estas formas de onda complejas. El RMT logra esta precisión gracias al reloj base de la ESP32 de 80MHz, que ofrece una resolución de 12.5ns. Al usar un divisor de 8, por ejemplo, el RMT obtiene una granularidad de 100ns, lo que lo hace perfectamente adecuado para generar los pulsos de ~400ns que requieren los WS2812, una tarea inviable de lograr de forma fiable mediante lógica de software.

Para que estos actuadores realicen acciones coherentes basadas en los datos de los sensores, se necesita un componente central que orqueste todo el proceso: el microcontrolador.

Usando potenciómetro, sensor DHT22 y encoder.

La placa de desarrollo UTN viene integrada con un sensor DHT22 que permite monitorear temperatura y humedad relativa de forma precisa y sencilla a un bajo precio. La salida suministrada es de tipo digital, no requiriéndose utilizar entradas analógicas. Dicho valor se lee mediante el **pin 33**. Por su parte el potenciómetro está integrado como entrada analógica en el **pin 32**.



> **Caso práctico 04**: desarrollar un sketch para regular la intensidad del LED verde utilizando como entrada los valores obtenidos del potenciómetro (o divisor de voltaje). Los valores deberán ser visualizados en el módulo OLED utilizado en la práctica anterior, indicando valor de lectura y voltage resultante (valor comprendido entre[0;3.3]v) que será enviado como salida al led.

Para ver el código de la solución acceder a esta carpeta: Práctico 04

> **Caso práctico 05**: desarrollar un sketch que permita leer los valores de Humedad y Temperatura relativas mediante el sensor DTH22 integrado. En todo momento los valores deben ser visualizados en el módulo OLED. En caso de los valores de temperatura leídos por el sensor superen los 20 °C, se deberá encender el led verde, cuando no llegue a ese valor de lectura deberá apagarse automáticamente.

Para ver el código de la solución acceder a esta carpeta: Práctico 05

> **Caso práctico 06**: desarrollar un sketch que permita mostrar por el diplay los pasos decrecientes (en sentido horario) o decrecientes (en sentido contrario) leidos desde el encoder. En caso de presionar el botón se deberá encender/apagar la luz led respectivamente. Se deberán mostrar los datos:

- Pasos
- Sentido de giro: RELOJ/NO-RELOJ
- Estado LED: ON/OFF

Para ver el código de la solución acceder a esta carpeta: Práctico 06

El Ciclo de Control: La Relación Sensor – Controlador – Actuador

El paradigma fundamental que gobierna el funcionamiento de los sistemas embebidos interactivos es el ciclo "Sensor-Controlador-Actuador". Este modelo describe un proceso de retroalimentación continuo donde el sistema percibe su entorno, procesa la información y actúa en consecuencia para alcanzar un objetivo. Este bucle se puede desglosar en tres pasos secuenciales que se repiten constantemente:

1. Percepción (Sensor): Un sensor mide una variable física del entorno y la convierte en una señal eléctrica.
2. Procesamiento (Controlador): El microcontrolador lee la señal del sensor, la compara con un estado deseado y calcula la acción correctiva necesaria.
3. Acción (Actuador): El controlador envía una señal a un actuador, que realiza una acción física para modificar el entorno.

Esta dinámica está intrínsecamente vinculada a los sistemas de control a lazo cerrado. En este tipo de sistemas, la información de salida (lo que el sensor mide) se retroalimenta (feedback) a la entrada del controlador. Este utiliza el feedback para ajustar de forma continua la salida del actuador, con el fin de minimizar el error entre el estado actual y el estado deseado. Este bucle permite que los sistemas se autorregulen y se adapten a las perturbaciones externas de forma autónoma.

Para ilustrar este concepto, consideremos un escenario práctico hipotético construido exclusivamente con los componentes mencionados anteriormente:

Imaginemos una plataforma de estabilización automática. Un sensor de movimiento MPU-6050 está montado en la plataforma para medir constantemente su inclinación. La ESP32 (el controlador) lee estos datos de orientación. La lógica programada en la ESP32 compara la inclinación actual con la posición objetivo (perfectamente horizontal). Si detecta una desviación, calcula el ajuste necesario y envía señales de control a dos servomotores (los actuadores) que están conectados a la base de la plataforma. Los servomotores giran en la dirección opuesta a la inclinación para corregirla. El sensor MPU-6050 detecta este cambio, y el ciclo se repite continuamente para mantener la estabilidad, cerrando así el bucle de control.

Aunque este ciclo define la lógica operativa, la integración física de los componentes a menudo presenta desafíos prácticos, especialmente a nivel de interfaces eléctricas y gestión de la potencia.

Gestión de Interfaces y Potencia: Aislamiento y Manejo de Carga

La integración de sensores, actuadores y microcontroladores en un sistema funcional requiere una gestión cuidadosa de las diferentes tensiones y corrientes eléctricas. Ignorar estos aspectos puede llevar a un funcionamiento errático, lecturas de sensores imprecisas o, en el peor de los casos, a dañar permanentemente los componentes. Dos de los desafíos más comunes son el manejo de cargas de alta potencia y la compatibilidad entre diferentes niveles de voltaje lógico.

Un microcontrolador como la ESP32 opera a bajo voltaje (3.3V) y sus pines GPIO solo pueden suministrar una corriente muy limitada. Esto es suficiente para encender un pequeño LED, pero no para alimentar dispositivos que demandan más potencia, como motores o tiras de LEDs. Intentar alimentar una carga pesada directamente desde un pin GPIO puede destruir el microcontrolador.

La solución es utilizar un componente de conmutación. Un transistor o relé, por ejemplo, puede actuar como un interruptor electrónico. Al conectar su base a un pin GPIO de la ESP32, una pequeña corriente de control puede gestionar un flujo de corriente mucho mayor que circula entre el colector y el emisor del transistor. Esto permite que una señal de baja potencia de la ESP32 encienda o apague de forma segura un dispositivo

externo alimentado por una fuente de mayor voltaje o corriente. Este principio es análogo al funcionamiento de un relé, pero en estado sólido y con mayor velocidad.

Entorno de trabajo para Esp32

Para comezar a trabajar con ESP32 es necesario instalar:

- Arduino IDE como herramienta de desarrollo.
- Driver de la placa instalado en Arduino IDE.

Para obtener instrucciones paso a paso de cómo instalar el Entorno de Desarrollo (IDE) de Arduino se sugiere visitar el enlace: https://www.arduino.cc/en/Guide, donde encontrará una guía con los pasos necesarios para instalar este software, así como también documentación de referencia y ejemplos. Actualmente se encuentra en su versión 2.X que a diferencia de la versión anterior se incorporar características muy requeridas de desarrollo como el autocompletado, la integración con los repositorios de código GIT y el debbuger, entre otros. Para mayor información visitar este video: https://www.youtube.com/watch?v=7gymFdvJQ2s.

Otra alternativa es trabajar con el Editor Visual Studio Code e instalar las extensión particulares para hacer desarrollos sobre Arduino. https://www.luisllamas.es/arduino-visual-studio-code/

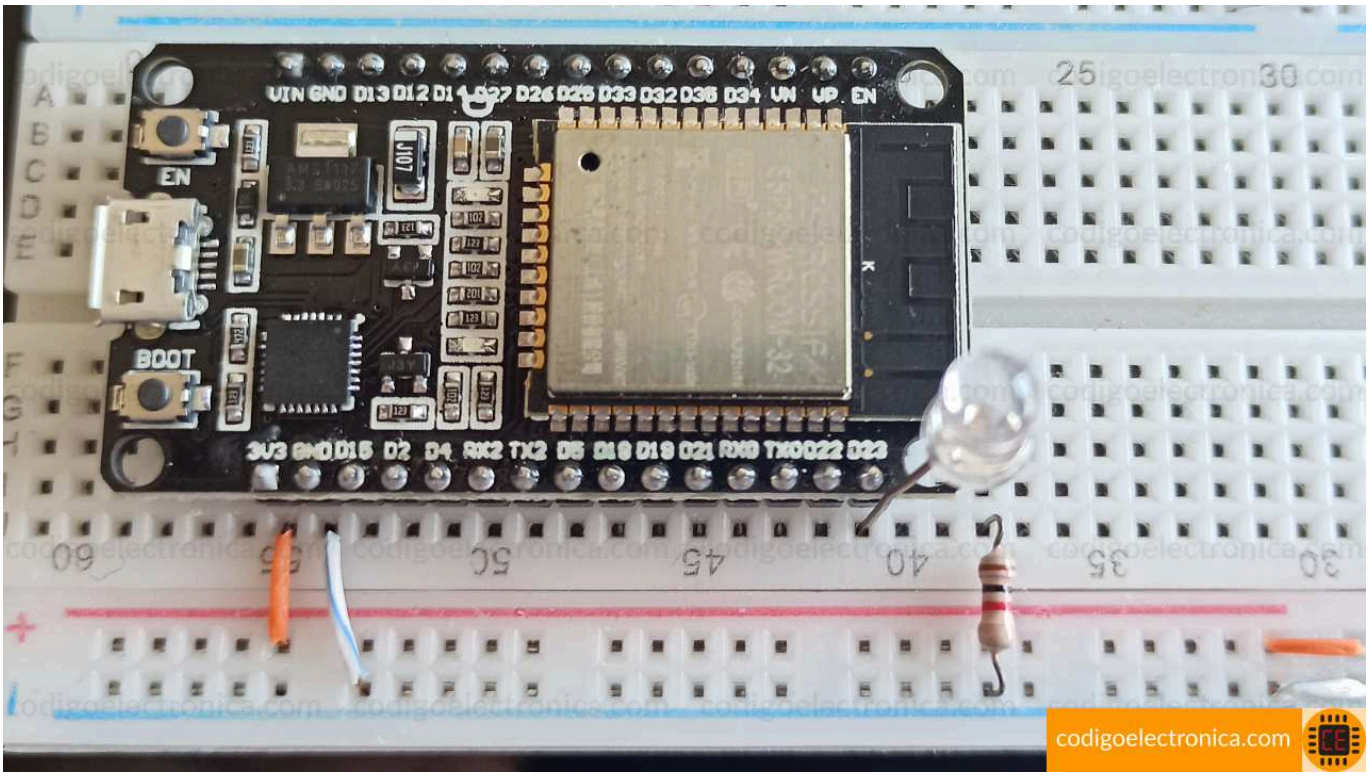
Una vez instalado **Arduino IDE** es necesario instalar el controlador de la placa. Para ello se deja el siguiente enlace con los pasos necesarios: https://www.electrosoftcloud.com/esp32-configuracion-y-primeros-pasos/. Es también de gran ayuda ver el siguiente video que explica cómo averiguar qué driver es necesario instalar para Win10 según el modelo de la Esp32: https://www.youtube.com/watch?v=JmDxP4O4Trk. En la descripción del video están los enlaces a los drives mencionados.

Primer ejemplo: Blink

> Para comenzar con un primer ejemplo de aplicación se va trabajar con el clásico ejemplo de encender y apagar un led usando el esp32 como controlador.

Para comenzar en necesario disponer de los siguientes materiales:

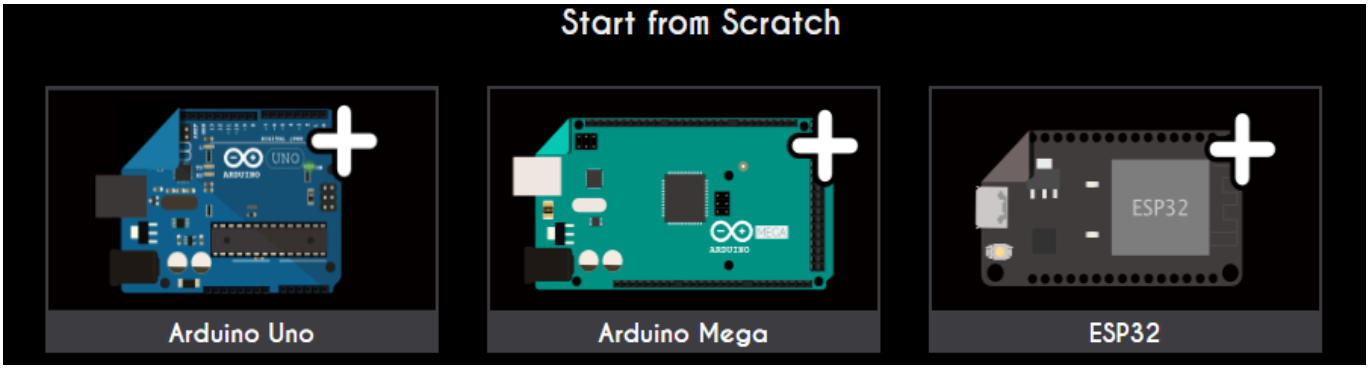
- Placa Esp32
- Led
- Resistencia de 1k (opcional ya que todos los pines de la placa tiene una resitencia Pull-Down y Pull-Up integradas)



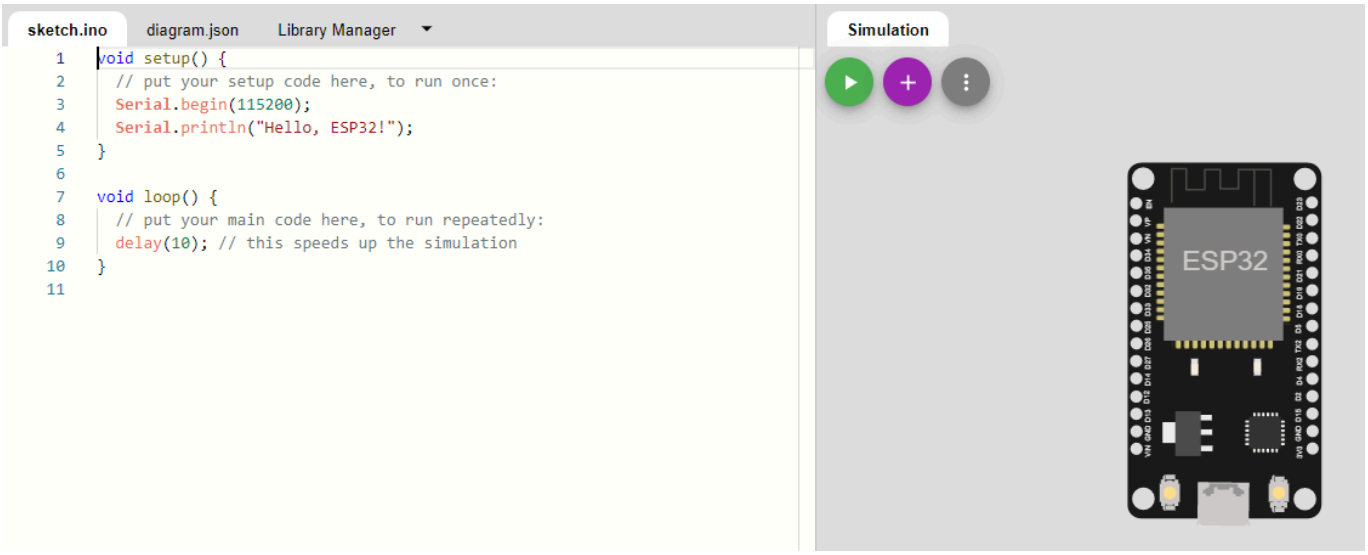
Diseñar lógicamente el circuito con WokWi

Wokwi es un simulador de electrónica en línea que permite simular desarrollos sobre Arduino, ESP32 y muchas otras placas, piezas y sensores populares. Para comenzar a utilizarlo solo es necesario crear una cuenta de usuario clásica a partir de un correo electrónico personal.

Una vez creado el usuario, en el home del sitio se selecciona la placa Esp32 desde la opción *start from Scratch*:



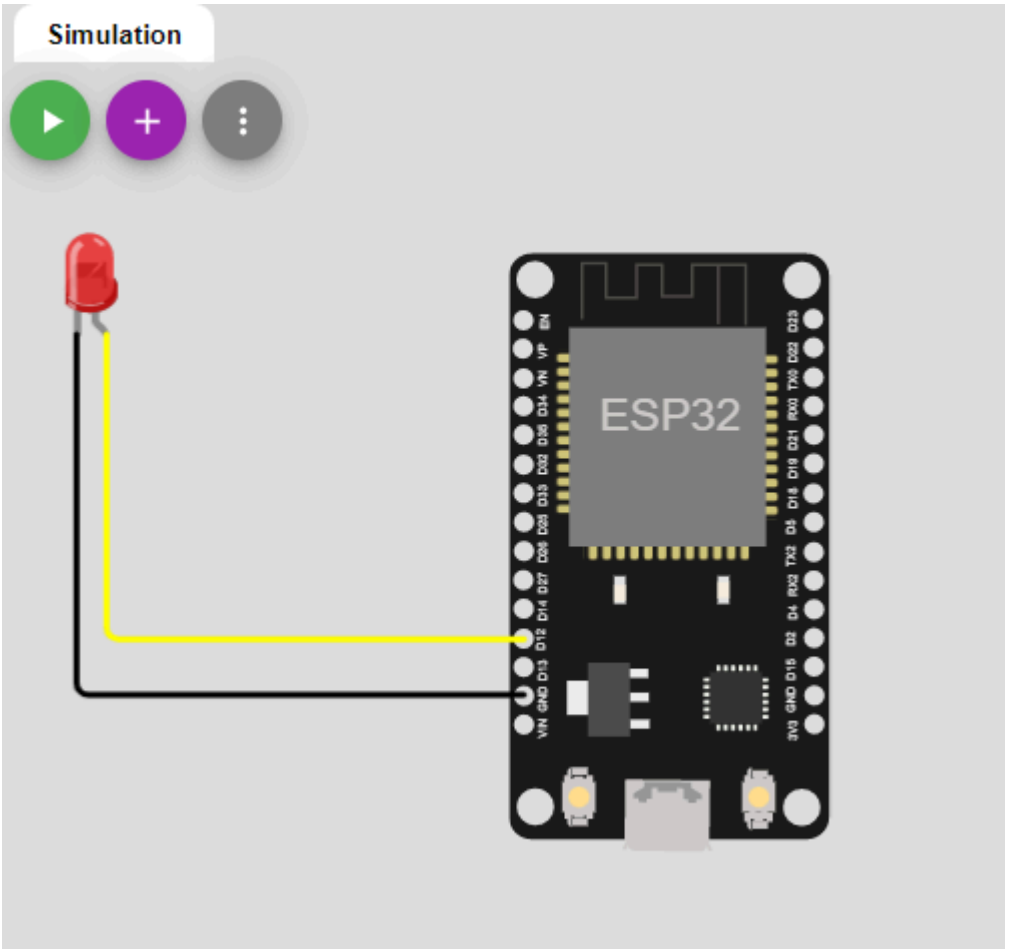
Una vez seleccionada la placa se muestra:



En la parte izquierda se habilita un editor para escribir las líneas de código C/C++, al igual que en Arduino IDE. En la parte derecha se muestra gráficamente la placa y es posible incorporar mediante la opción (+) los dispositivos electronicos como leds, pulsadores, potenciometros, pantallas, etc. Luego se conectan los pines de la placa a los dispositivos agregados y la herramienta actualiza la información de conexionados mediante un archivo JSON que puede gestionarse desde la solapa *diagram.json*. Es posible definir identificadores, colores y posiciones de los elementos dibujados.

De ser necesario, la pestaña *Library Manager* permite cargar librerías específicias de desarrollo según los componentes agregados al desarrollo.

Retornando al ejemplo del **Blink**, el diagrama Wokwi resulta:



Donde el conector cátodo (-) del led se conecta a la señal **GND** del Esp32, mientras el pin **D12** se conecta con el ánodo o conector positivo (+).

Finalizado el conexionado de la placa, se escribe el código que será instalado y ejecutado. Para compilar, instalar y ejecutar el sketch en la Esp32 se selecciona la opción **Play (>)**.

Desde la opción **Guardar** es posible descargar tanto el código en formato .ino como el archivo de diagrama en formato .json con la configuración de los dispostivos y la placa.

Para ver el código de la solución acceder a esta carpeta: Blink

Integración Wokwi con Platformio desde Visual Studio Code

Para llevar a cabo las prácticas de este módulo, combinaremos el poder de PlatformIO y Wokwi, dos herramientas fundamentales en el desarrollo moderno de sistemas embebidos. Mientras que PlatformIO (integrado en Visual Studio Code) nos proporciona un entorno profesional para la gestión de proyectos, librerías y compilación de código, Wokwi actúa como un simulador de hardware en tiempo real. Esta integración nos permite escribir, compilar y probar nuestros proyectos con microcontroladores directamente desde el navegador o la computadora, sin necesidad de contar inmediatamente con los componentes físicos. Para obtener un paso a paso cómo configurar y sacar el máximo provecho a este flujo de trabajo se sugiere la lectura del siguiente pdf Entorno de trabajo integrado.

Programación en Arduino

Lenguaje C/C++

- Creado por Dennis Ritchie en los laboratorios de AT&T entre 1969 y 1973
- Para escribir el SO Unix en un Lenguaje de Alto Nivel e independiente del hardware
- Basado en el B de Ken Thompson
- En 1978 Ritchie junto a Bryan Kernigan publican el libro: “El lenguaje de programación C”
- A partir de dicho libro surge el lenguaje conocido como “el C de Kernigan y Ritchie” o simplemente “K&R C”
- A mediados de los 80 Bjarne Stroustrup crea el Lenguaje C++

> Para programar un Arduino, el lenguaje estándar es C++, aunque es posible programarlo en otros lenguajes. No es un C++ puro sino que es una adaptación que proveniente de avr-libc que provee de una librería de C de alta calidad para usar con GCC en los microcontroladores AVR de Atmel y muchas funciones específicas para los MCU AVR de Atmel.

Fundamentos

- Lenguaje de programación estructurada + POO
- Originalmente concebido como de alto nivel, hoy de medio/bajo nivel en comparación con los lenguajes actuales
- Lenguaje Compilado, es decir que se parte del código fuente se compila en una plataforma y se obtiene un ejecutable para esa plataforma
- Manejo libre y a cargo del programador de la memoria
- Lenguaje fuertemente tipado.

Comentarios

Dos posibilidades para definir comentarios en el código de un sketch:

- Comentario de una sola línea: //comentario...
- Comentario en más de una línea : /.../

Tipo de datos

El ambiente Arduino es realmente C++, con bibliotecas de soporte, y además asume algunos parámetros relativos al microcontrolador, para simplificar el proceso de programación. C++ define una cantidad de tipos de datos distintos.

A continuación se listan tipos de datos comúnmente utilizados en el ambiente **Arduino**, con el tamaño de memoria de cada uno entre paréntesis.

- **boolean** (8 bit)- lógico simple verdadero/falso.
- **byte** (8 bit)- número sin signo entre 0 y 255.
- **char** (8 bit)- número con signo, entre -128 y 127. En algunos casos el compilador intentará interpretar este tipo de dato como un caracter, lo que puede generar resultados inesperados.
- **unsignedchar** (8 bit)- lo mismo que 'byte'; si es que eso es lo que necesitas, deberías usar 'byte', para que el código sea más claro.
- **word** (16 bit)- número sin signo entre, 0 y 65535.
- **unsignedint** (16 bit)- lo mismoque 'word'. Utiliza 'word' por simplicidad y brevedad.
- **int** (16 bit)- número con signo, entre -32768 y 32767. Este tipo es el más usado para variables de propósito general en Arduino, en los códigos de ejemplo que vienen con el IDE.
- **unsignedlong** (32 bit)- número sin signo entre 0 y 4294967295. Este tipo se usa comúnmente para almacenar el resultado de la función millis(), la cual retorna el tiempo que el código actual ha estado corriendo, en milisegundos.
- **long** (32 bit)- número con signo, entre -2,147,483,648 y 2,147,483,647.
- **float** (32 bit)- número con signo, entre 3.4028235E38 y 3.4028235E38. El Punto Flotante no es un tipo nativo en Arduino; el compilador debe realizar varios saltos para poder hacerlo funcionar. Evítalo siempre que te sea posible. Hablaremos de eso más tarde; En una fecha próxima se publicará un tutorial más riguroso en el uso genérico de la matemática de punto decimal en Arduino.

Variables simples y estructuras de control

Al igual que C++ es posible definir variables simples y estructuradas; y combinarlas con estructuras de control condicionales y repetitivas. El detalle completo de variables, operadores, funciones, punteros y estructuras de control está disponible en la documentación de referencia de Arduino

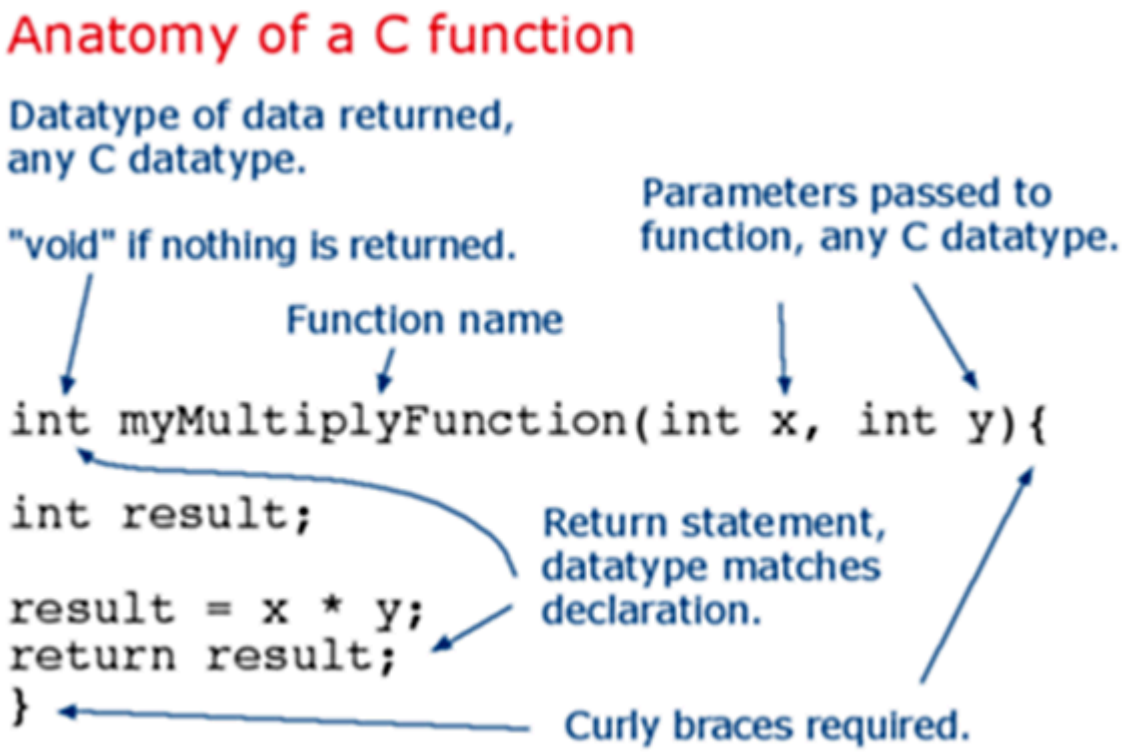
Arreglos

Un array (**o arreglo**) es un conjunto estático de valores (todos del mismo tipo) a los que se accede con un número índice. Cualquier valor puede ser recogido haciendo uso del nombre de la variable y el número del índice. El primer valor del arreglo es el que está indicado con el índice 0, es decir el primer valor del conjunto es el de la posición 0. Un array tiene que ser declarado y opcionalmente asignados valores a cada posición antes de ser utilizado.

Tener en cuenta:

- Para definir: **tipo_de_dato** nombre[size]; O también **tipo_de_dato** nombre[] = {val, val2,....,valn};
- Para acceder: nombre[índice]
- Las cadenas tienen un tratamiento especial, pero se manejan como arreglos de caracteres. Por ejemplo: char message[6] = "hello";

Uso de funciones



Parámetros por Valor

Pasar parámetros por valor o copia, las modificaciones realizadas en la función no se mantienen al terminar la misma. Picture02

Parámetros por referencia

Los parámetros pasados por referencia pueden ser modificados en la función y dicha modificación sobrevive al final de la misma.

- Se agrega operador & en la definición del prototipo de la función int myFuncion(int &p1, int p2)
- Otra manera de lograrlo es utilizar punteros:

  int myFunction(int * p1, int p2)

Constantes y funciones basicas de E/S

Para **E/S digitales**

Función	Descripción
pinMode(pin, modo)	Configura el pin especificado para que se comporte como una entrada (INPUT) o una salida (OUTPUT). Adicionalmente es posible indicar INPUT_PULLUP, INPUT_PULLDOWN para entradas con resistencias integradas PULL_UP o PULL_DOWN respectivamente
digitalWrite(pin, valor)	Escribe un valor HIGH o un LOW en un pin digital.
digitalRead(pin)	Lee el valor de un pin digital especificado, ya sea HIGH o LOW.

Para **E/S analógicas**

Función	Descripción
analogRead(pin)	Lee el valor del pin analógico especificado. Las placas Arduino contienen un convertidor analógico a digital multicanal de 10 bits. Esto significa que mapeará los voltajes de entrada entre 0 y el voltaje de funcionamiento (5 V o 3,3 V) en valores enteros entre 0 y 1023. Puntualmente en Esp32 es posible leer lecturas de 0 a 3.3V asignando un valor comprendido entre 0 y 4095.
analogWrite(pin,valor)	Escribe un valor analógico (onda PWM) en un pin. Se puede usar para encender un LED con diferentes brillos o impulsar un motor a varias velocidades.
analogReference()	Configura el voltaje de referencia utilizado para la entrada analógica (es decir, el valor utilizado como la parte superior del rango de entrada). En Esp32 el valor por defecto es 3.3V

Para **comunicación Serie**

Función	Descripción
Serial.begin(speed)	Establece la velocidad de datos en bits por segundo (baudios) para la transmisión de datos en serie. Para comunicarse con Serial Monitor, asegúrese de usar una de las velocidades en baudios enumeradas en el menú en la esquina inferior derecha de su pantalla. Un segundo argumento opcional configura los bits de datos (Serial.begin(speed, config)), paridad y parada. El valor predeterminado es 8 bits de datos, sin paridad, un bit de parada.
Serial.read()	Lee los datos seriales entrantes.
Serial.print(val, [format])	Imprime datos en el puerto serial como texto ASCII legible por humanos. Este comando puede tomar muchas formas. Los números se imprimen utilizando un carácter ASCII para cada dígito. Los flotantes se imprimen de manera similar como dígitos ASCII, por defecto con dos decimales. Los bytes se envían como un solo carácter. Los caracteres y las cadenas se envían tal cual.
Serial.println(val, [format])	Imprime datos en el puerto serie como texto ASCII legible por humanos seguido de un carácter de retorno de carro (ASCII 13 o '\r') y un carácter de nueva línea (ASCII 10 o '\n'). Este comando toma las mismas formas que Serial.print() .

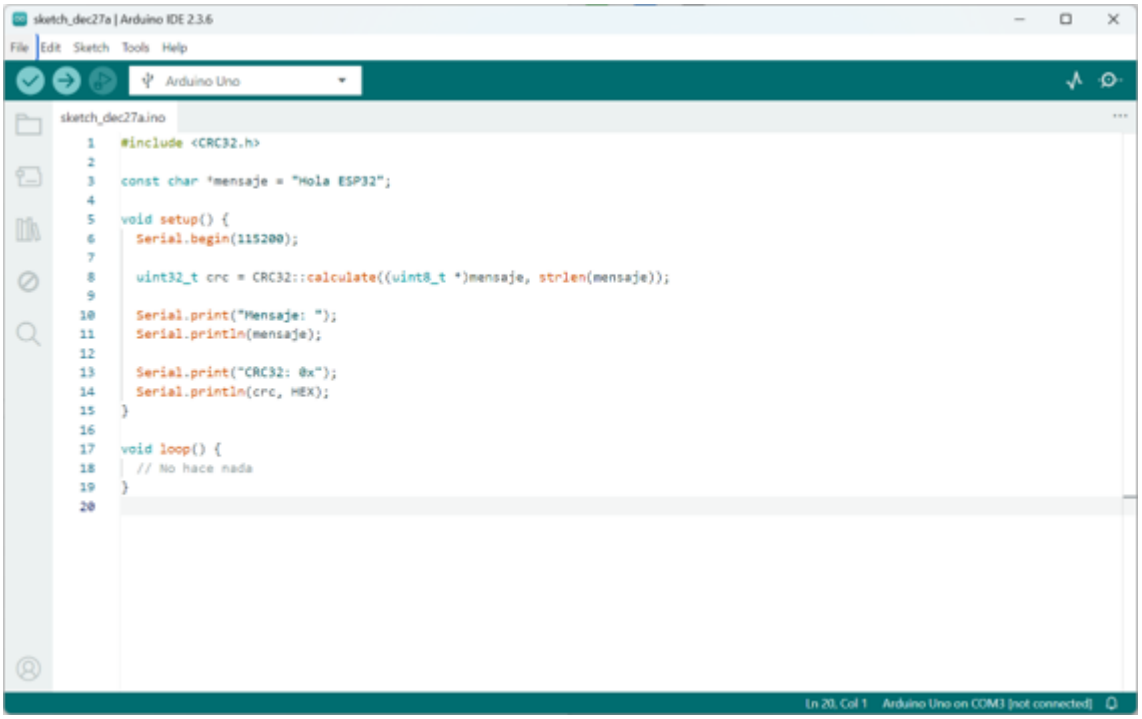
Función	Descripción
Serial.available()	Obtiene la cantidad de bytes (caracteres) disponibles para leer desde el puerto serie. Estos son datos que ya llegaron y se almacenaron en el búfer de recepción en serie (que contiene 64 bytes).

Constantes más usadas:

Constantes	Descripción
INPUT/OUTPUT	Permite definir entradas o salidas mendiante la función pinMode().
HIGH (5V)/LOW (0V)	Permiten indicar un valor Alto o Bajo de tensión dependiendo la configuración del pin.
LED_BUILTIN	La mayoría de las placas Arduino tienen un pin conectado a un LED integrado en serie con una resistencia. La constante LED_BUILTIN es el número del pin al que está conectado el LED integrado. La mayoría de las placas tienen este LED conectado al pin digital 13. Para el caso de los Esp32 este valor corresponde a 2

Librerías

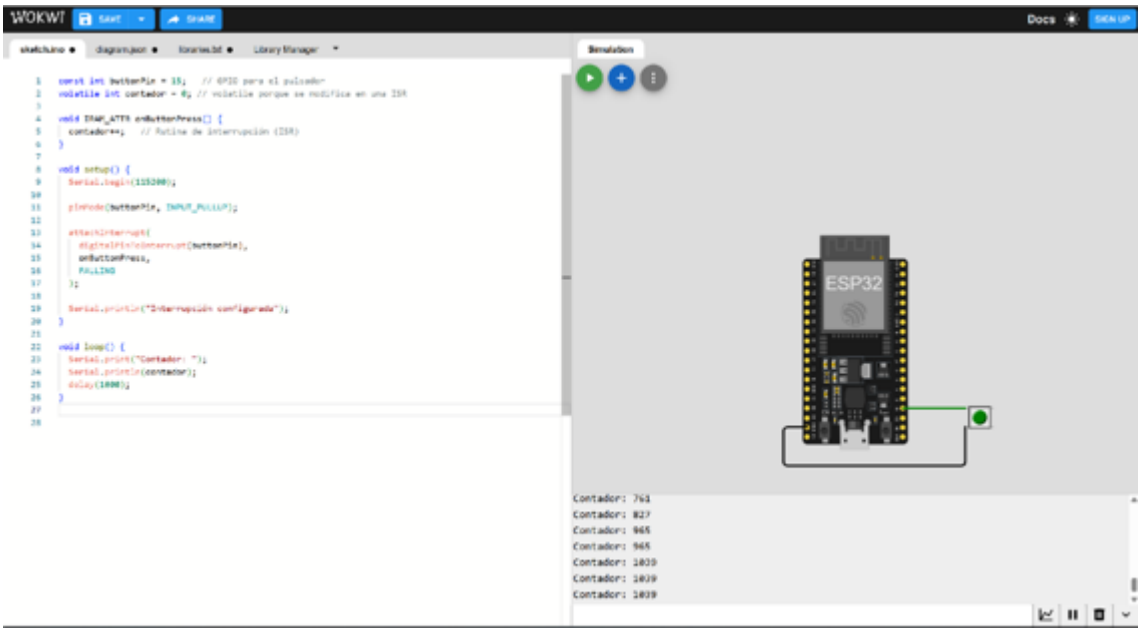
Las librerías permiten encapsular funcionalidades complejas en módulos reutilizables, simplificando el desarrollo de software embebido. En el entorno Arduino para ESP32, las librerías abstraen el acceso a periféricos, protocolos de comunicación y servicios del sistema operativo subyacente, reduciendo la necesidad de interactuar directamente con registros de bajo nivel. Desde una perspectiva educativa y de ciberseguridad, el uso de librerías debe ir acompañado de un análisis crítico. Comprender qué hace internamente una librería, qué recursos utiliza y cómo gestiona errores es fundamental para evitar dependencias opacas que puedan introducir vulnerabilidades o comportamientos inesperados.



Interrupciones por Hardware

Las interrupciones por hardware son mecanismos que permiten al microcontrolador responder de forma inmediata a eventos externos, como el cambio de estado de un pin GPIO, sin necesidad de consultar constantemente su estado. Cuando ocurre el evento, la ejecución normal del programa se suspende temporalmente y se ejecuta una rutina de servicio de interrupción (ISR).

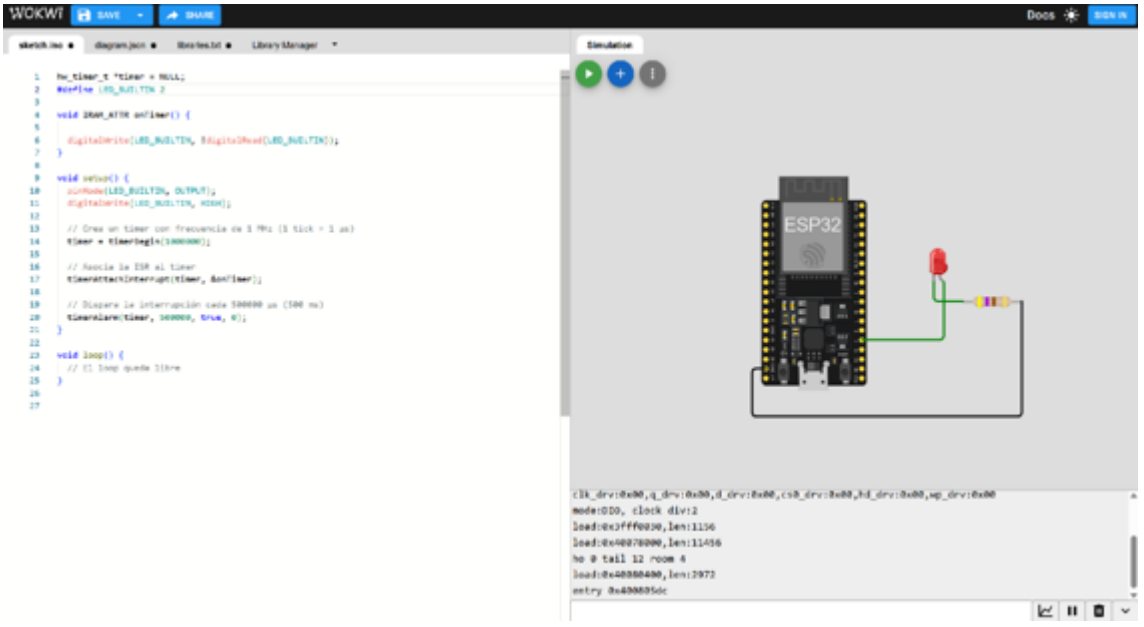
En el ESP32, las interrupciones por hardware son una herramienta potente, pero deben utilizarse con cuidado. Las ISR deben ser breves, no bloqueantes y evitar el uso de funciones que no sean seguras en contexto de interrupción, ya que un mal diseño puede afectar la estabilidad del sistema completo.



Interrupciones por Timers

Las interrupciones generadas por timers permiten ejecutar código de manera periódica con una precisión temporal superior a la obtenida mediante retardos por software. El ESP32 dispone de temporizadores de hardware que pueden configurarse para generar interrupciones a intervalos definidos.

Este mecanismo es clave para tareas como muestreo periódico de sensores, generación de señales o control de tiempo real. En sistemas basados en FreeRTOS, los timers deben integrarse adecuadamente con las tareas del sistema para evitar conflictos de temporización o sobrecarga del procesador.



Conexión a Internet



Conexión Wifi

Introducción

Es muy normal el poseer dispositivos que pueden conectarse a Internet, como lo es un teléfono móvil, una laptop, una televisión, etc. Hay muchos casos en los cuales, se necesita hacer el envío de información hacia alguna plataforma alojada en la Nube. Lo necesario para realizar esto es conectarse a **Internet**.

En esta sección se realizará la conexión de la tarjeta ESP32 hacia una red WiFi aprovechando que nativamente el chip cuenta con conectividad WiFi. Esta última es compatible con 802.11 b/g/n en la banda de los 2.4GHz, alcanzando velocidades de hasta 150 Mbits/s. También incluye comunicación Bluetooth compatible con Bluetooth v4.2 y Bluetooth Low Energy (BLE) que serán mencionados a posterior.

Configurar una conexión Wifi

Para conectar la tarjeta Esp32 a una red inalámbrica es necesario conocer como mínimo los siguientes datos:

- **SSID**: es el nombre público que identifica una red local inalámbrica, es decir, una WLAN. Son las siglas de Service Set Identifier.
- **PASSWORD**: Clave de seguridad de la red.

Estos datos pasan a ser constantes del sketch cuyos valores dependerán de la red a la que se conecta la placa. A continuación se muestra el código necesario:

```
#include <WiFi.h>

const char* SSID = "UNA_RED";
```

```
const char* PASS = "UN_PASS";

void setup() {
   Serial.begin(9600);
   Serial.println("");
   WiFi.begin(SSID, PASS);
   while (WiFi.status() != WL_CONNECTED) {
       delay(500);
       Serial.print(".");
   }
   Serial.println("");
   Serial.println("WiFi conectado");
   Serial.println(WiFi.localIP());
   server.begin();  // iniciamos el servidor
}
```

El funcionamiento del código anterior se explica a continuación:

Como primer paso, se debe llamar a la librería WiFi.h, que se encuentra instalada en el IDE de Arduino por defecto (No es necesario instalar).

```
#include <WiFi.h>
```

Ahora, se definen como char constante el nombre y la contraseña de la red WiFi a la cual se realiza la conexión.

```
const char* SSID = "UNA_RED";
const char* PASS = "UN_PASS";
```

Se define en la función **setup()** el inicio del terminal Serial a una velocidad de baudios determinada, y se inicia la conexión WiFi con el comando **WiFi.begin()**, añadiendo los nombres de las variables mencionadas (SSID y PASS). Adicionalmente se añade un retardo de 2 segundos.

```
void setup() {
 /*Iniciamos el terminal Serial para la escritura de >algunos parámetros */
 Serial.begin(9600);
 /*Iniciamos la conexión a la red WiFi definida*/
 WiFi.begin(SSID, PASS);
 delay(2000);
```

Se añade un ciclo While que tiene como función imprimir puntos (“.”) mientras se realice la conexión con la red WiFi. Una vez conectado, dejarán de aparecer esos caracteres.

```
  Serial.print("Se está conectando a la red WiFi denominada ");
  Serial.println(ssid);
  while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
```

Cuando la conexión se haya realizado, se imprimirá en el monitor Serial “WiFi Connected”, junto a la dirección IP que se le asignó al ESP32. Cabe mencionar que no se contempla el uso de Loop () debido a que, la conexión se realiza una única vez, por lo que no es necesario realizar un bucle infinito de conexiones.

```
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
```

Servidor Web con Esp32

Introducción

Un servidor web con ESP32, ¿es posible alojar un servidor Web en la tarjeta?, ¿para qué?. La respuesta es sencilla: para poder acceder de manera remota a la placa mediante el protocolo de comunicación HTTP sin necesidad de tener hardware especializado. Esto permite tener el control remoto de la Esp32 desde cualquier punto de la red desde el que se tenga acceso. Esto último implica dos cosas:

- tanto el cliente Web como el esp32 estan en la misma red (se accede directo por IP local asignada a la placa)
- la tarjeta tiene asignada una IP fija y el cliente accede desde cualquier punto por Internet. Para más información consultar este link.

¿Cómo funciona un Servidor Web?

Un servidor Web es un proceso dedicado capaz de responder mensajes mediante el protocolo HTTP enviados desde un cliente. Típicamente los clientes Web más usados son los navegadores Web como Google Chrome, Firefox, etc.

Comunmente cuando se escribe la **URL** (cadena que identifica unívocamente a un recurso del servidor) desde un navegador lo que este hace es enviar una solicitud HTTP al servidor WEB (mediante un método específico del protocolo llamado **GET**). El trabajo se traslada al servidor WEB que debe de poder procesar esa solicitud.

Por ejemplo, si se escribe una URL como **http://192.168.1.125/ledon** en el navegador de un PC o móvil, el navegador envía una solicitud HTTP al ESP32 (identificado como un nodo más de la red) para que procese esta solicitud, encendia un LED y devuelva como respuesta una página web (contenido HTML) al navegador mostrándole el estado del LED: ON.

Modos de funcionamiento

El ESP32 con la librería Wifi.h nos posibilita trabajar de varias maneras diferentes. No solo la típica conexión al AP (punto de acceso) de tu router (como cualquier PC o móvil), sino que también permite una conexión directa entre ESP32 y los clientes sin pasar por el router, ni por su punto de acceso Wifi.

- **Modo Estación (STA)**: su comportamiento que como cualquier otro dispuesto en vuestra casa, móvil, PC o tablet.
- **Modo de Punto de acceso (AP)**: en este modo de operar, el ESP32 crea su propia red Wifi, permitiendo conexiones máquina a máquina, estableciendo una comunicación directa entre dos o más estaciones (PC, móvil, Tablets) sin un punto de acceso que enrute el tráfico.
- Existe un tercer modo que es un modo mixto, es decir, es un híbrido que se comporta de los dos modos anteriores.

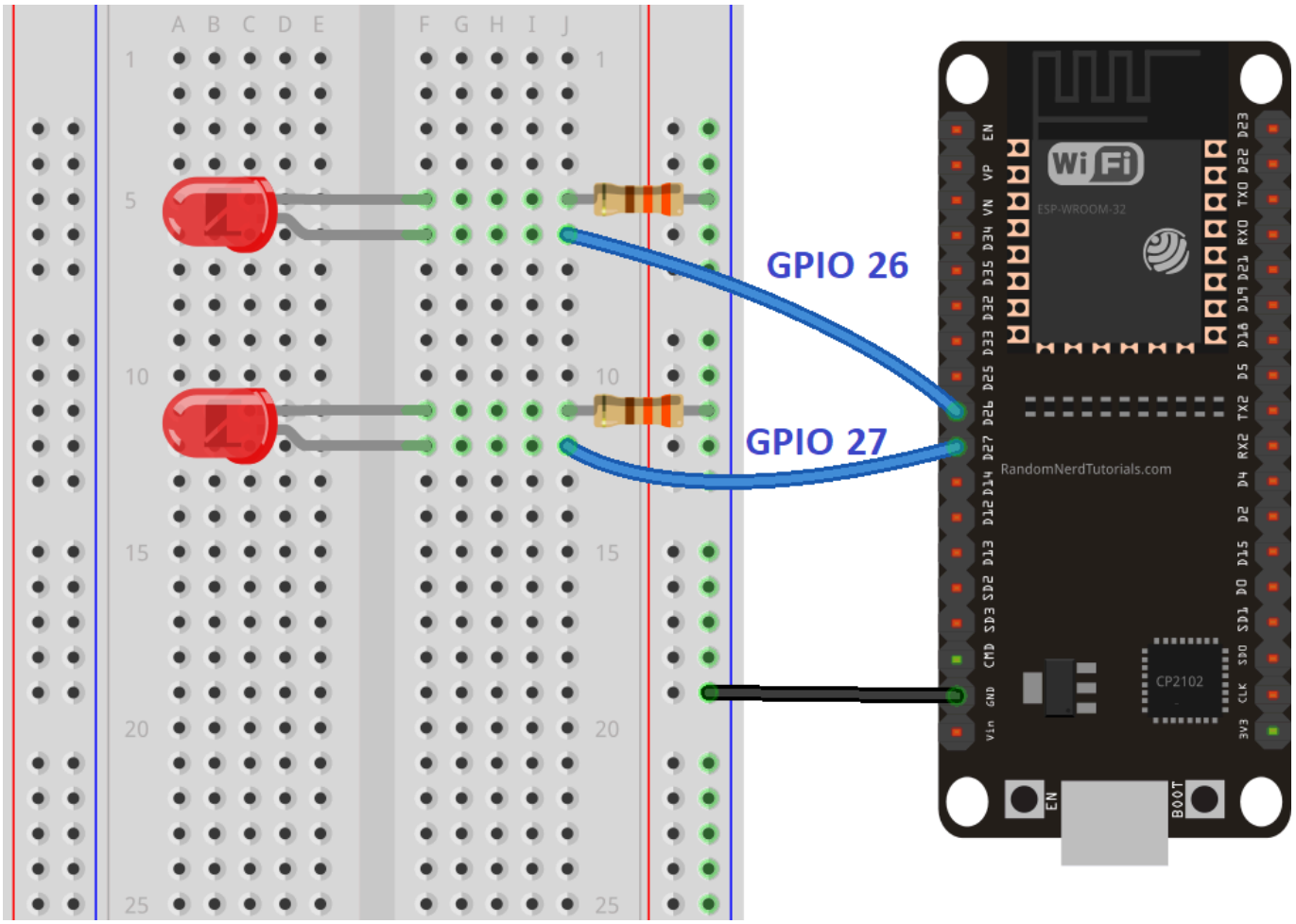
Cualquiera de estos modos de operar son posibles con ESP32, la diferencia radica en decidir como se quiere que se comporte el ESP32.

Primer servidor Web

Para un primer ejemplo se creará un servidor web independiente con un ESP32 que controla las salidas (dos LED) mediante el entorno de programación Arduino IDE. El servidor web responde a dispositivos móviles y se puede acceder a él con cualquier dispositivo que funcione como navegador en la red local. Se mostrará cómo crear el servidor web y cómo funciona el código paso a paso.

Circuito del proyecto

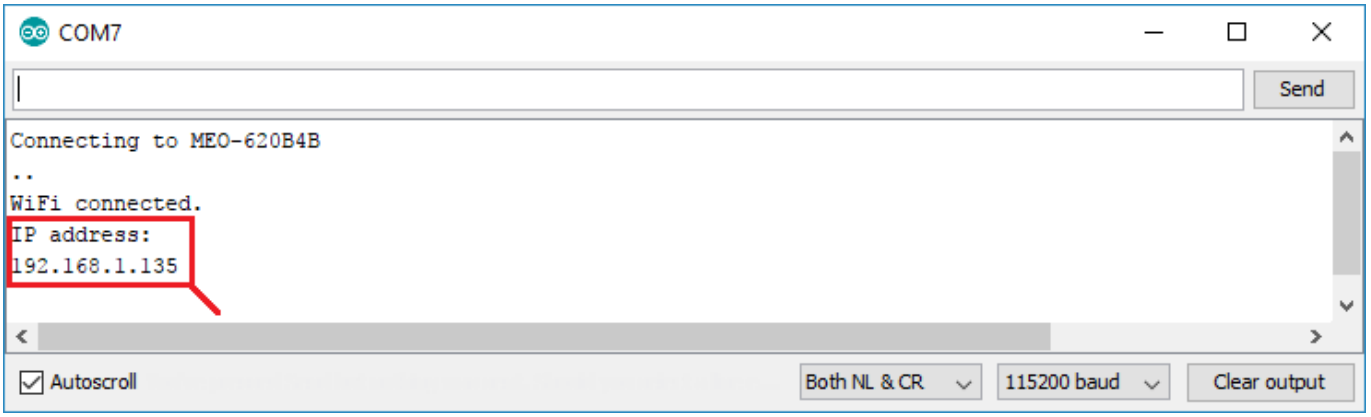
Se conectan dos LED al ESP32 como se muestra en el siguiente diagrama esquemático: un LED conectado a **GPIO 26**, y el otro para **GPIO 27**.



Código en Arduino IDE

El código de este primer servidor esta Práctico 07.

Para que el código anterior funcione es necesario obtener el SSID y clave de la red local a la se va a conectar tanto el Esp32 como el movil o pc desde donde se quiera hacer la prueba. Luego de definir dichos valores como constantes en el sketch es necesario subir el código a la placa y obtener desde el Monitor Serial la dirección IP asignada a la tarjeta (tal como se muestra a modo de ejemplo en la siguiente imagen).



Consideraciones de funcionamiento:

> Una vez definidos los valores de las constantes SSID y PASS >lo siguiente es crear un servidor en un puerto determinado por >ejemplo puerto 80 mediante la línea:

```
WiFiServer server(80);
```

> Luego se programa qué hacer cuando un nuevo cliente establece una conexión con el servidor web, tal como se muestra en la línea:

```
WiFiClient client = server.available();
if(client){
    //....
}
```

> El loop seguirá ejecutandose validando que la variable **client** no sea nula, en tal caso entiende que tiene un nuevo cliente que atender y comienza un nuevo ciclo mientras el cliente se encuentre conectado.

> Lo que sigue a continuación es el procesamiento típico de una solicitud HTTP: se lee byte a byte el header o cabecera de la solicitud hasta que se encuentra con un salto de línea o '\n'.En el header se encontrará tícamente con el método HTTP utilizado por el cliente (GET/POST) seguido de la URL que identifica el recurso solicitado. Con esta información es posible definir las acciones a tomar en función del estado de la placa.

> Por último se crea una cadena con el código HTML que será enviado al cliente como respuesta HTTP. Notar que el código incluye una sentencia break para salir del ciclo while() utilizado para el procesamiento de la petición.

Ejercitación

> Caso práctico 08:
>
> - Tomando el skectch del caso práctico 02, refactorizar su código para crear un servidor Web que permita encender el relay de manera remota utilizando la conexión WIFI a la red local, y dejar sin efecto el encendio mediante los comandos recibidos por el puerto serie. Para ver el código de la solución acceder a esta carpeta: Práctico 08

> Caso práctico 09:
>
> - Tomando el skectch del caso anterior, refactorizar su código para procesar mensajes recibidos desde un bot de Telegram. Deberá investigar cómo crear un bot y mediante su TOKEN ACCESS gestionar las comunicaciones con la placa de desarrollo. Librería sugerida: UniversalTelegramBot. Para ver el código de la solución acceder a esta carpeta: Práctico 9

Plataformas IoT

Introducción

Una plataforma de IoT es una parte integral de cualquier servicio basado en el Internet de las Cosas. Puede ayudarte a acelerar el tiempo de comercialización, minimizar el riesgo, reducir el costo de desarrollo y

ayudarte a llegar más rápido al mercado de los productos.

¿Qué es una plataforma de IoT?

Una plataforma de Internet de las cosas es un grupo de tecnologías que proporcionan los bloques de construcción para desarrollar un producto IoT. Las plataformas de IoT proveen la "infraestructura" que se usa para crear las características específicas de una solución.

Al asumir la funcionalidad no diferenciada, las plataformas de IoT ayudan a reducir el riesgo y el costo de desarrollo y a acelerar el tiempo de comercialización de un producto.

Cuando se habla de plataformas IoT, a menudo se lanza a la jerga técnica como protocolos de transporte, motores de reglas, repositorios de datos, etc. Si bien esas consideraciones son importantes y merecen una planificación cuidadosa, no ilustran claramente cómo una plataforma de IoT puede ayudar en el desarrollo de un producto IoT.

Funcionalidades clave

Las tareas clave de una plataforma IoT son:

- Adquirir datos del mundo real a través de sensores
- Analizar los datos localmente (edge computing)
- Conectarse a la nube para transmitir datos y recibir órdenes
- Almacenar los datos en la nube
- Analizar los datos en la nube para crear una visión
- Ordenar a las "cosas" que realicen tareas específicas basadas en la comprensión
- Presentar las ideas a los usuarios

Adicionalmente debería proporcionar servicios para:

- Realizar todas las operaciones de forma segura a través de la pila de tecnología IoT
- Identificar y gestionar todos sus dispositivos de IoT a escala

Plataforma Cloud IOT

Esta categoría de plataformas IoT proporciona los elementos básicos para su producto, incluyendo el consumo, el transporte, el almacenamiento, el análisis y la visualización de datos. Como su nombre lo indica, su objetivo es permitir el rápido desarrollo de su aplicación abstrayendo las complejidades de la construcción de una solución IoT.

Hay cientos de plataformas de Internet de las cosas en el mercado, por lo que puede ser desalentador averiguar cuál de ellas utilizar. Las plataformas de habilitación de aplicaciones incluyen:

- Plataformas industriales
- Plataformas de consumo
- Plataformas dirigidas a los desarrolladores
- Plataformas de alto nivel (arrastrar y soltar), buenas para la - creación de prototipos o MVPs
- Plataformas que se centran en verticales específicas
- Plataformas en el lugar vs. Edge vs. Cloud

Algunos de los principales actores en el espacio de las plataformas cloud IoT incluyen:

- Microsoft Azure IoT
- AWS IoT
- Google Cloud IoT Core
- ThingSpeak
- Arduino IoT Cloud

ThingSpeak

ThingSpeak es una plataforma abierta de aplicaciones, diseñada para permitir conectar personas con objetos. Se caracteriza por ser una plataforma Open Source con una API para almacenar y recuperar datos de los objetos usando el protocolo HTTP sobre Internet o vía LAN (Local Area Network).

Se trata de una plataforma basada en Ruby on Rails 3.0 (RoR), este es un framework de aplicaciones web de código abierto basado en Ruby, cuya arquitectura está basada en el Modelo Vista Controlador (MVC). Se caracteriza por su simplicidad a la hora de programar aplicaciones del mundo real, escribiendo menos código y con una configuración mucho más sencilla que otros frameworks. Otra de las características que hacen de RoR un framework perfecto para el desarrollo de aplicaciones es que permite el uso de meta programación, haciendo que su sintaxis sea más legible y llegue a un gran número de usuarios.

Características principales

Algunas características importantes de esta plataforma son:

- Dispone de una **API** de programación abierta. Un punto importante a la hora de desarrollar cualquier proyecto es encontrar un API disponible de forma sencilla para que el desarrollador tenga los mecanismos necesarios para el desarrollo de la aplicación. En este caso, ThigSpeak dispone de una API la cual está disponible en GitHub para su descarga en un servidor propio. Es totalmente abierta, por lo que también se puede modificar su código fuente original y así contribuir a la comunidad con nuevas características, un principio básico en toda plataforma Open Source.

- **Plugins**: para extender la funcionalidad del sitio también se nos brinda la oportunidad de desarrollar plugins. Estos nos ofrecen la posibilidad de crear aplicaciones de forma nativa en nuestra plataforma ThigSpeak. Soporta HTML, CSS y JavaScript como lenguajes de programación. Al igual que los canales los plugins pueden ser público o privados según sean nuestras necesidades.

- **Integración**: Uno de los puntos fuertes en cualquier plataforma IoT, es que permita una amplia integración con diversos dispositivos Hardware y software. En este caso ThingSpeak permite la integración de su plataforma con: Arduino/ Raspberry Pi/Móbiles/Aplicaciones web/Redes Sociales o Análisis de datos con MATLAB

- Otra característica es que dispone de un conjunto de apps para conectar con redes sociales, soportar comunicación con Web Services, calendarizar tareas o incluso definir reglas que disparen acciones ante ciertos valores de datos recibidos en un canal.

Chanels

Los canales almacenan todos los datos que recopila una aplicación ThingSpeak. Cada canal incluye ocho campos que pueden contener cualquier tipo de datos, más tres campos para datos de ubicación y uno para

datos de estado. Una vez que recopila datos en un canal, se utilizan las aplicaciones de ThingSpeak para analizarlos y visualizarlos.

Para crear canales es necesario contar primero con una cuenta en la plataforma. Una vez creado el usuario a partir de una mail validado, es posible crear canales para los proyectos.

Proyecto de aplicación

Para un primer proyecto de aplicación se propone generar un conjunto de valores aleatorios (con valores comprendidos entre 1 y 100) desde la placa Esp32 que podemos pensar en datos recolectados de un **sensor X** con un propósito específico y enviarlos a la plataforma para su visualización en tiempo real sobre un dashboard que grafique los valores enviados desde la placa.

Lo primero es crear un canal llamado **Random IoT Values** con un solo campo llamado **sensor**. Una vez confirmados los datos del canal, de la pestaña de **API Key** copiar los valores de: Channel ID y Write API Key. Dichos valores serán utilizados en el sketch de Arduino IDE para inicializar la librería de la plataforma.

El código del proyecto se encuentra en Práctico 10

Ejercitación

Caso Práctico 11: Diseñar una estación meteorológica desarrollada a partir de una placa ESP32 y un sensor DHT22. La información recolectada por el sensor es enviada a un servidor de ThingSpeak mediante Wifi.

El código del proyecto se encuentra en Práctico 11

Plataforma Arduino Cloud

IoT Arduino Cloud es una plataforma en la nube que permite, al igual que ThingSpeak, conectar y gestionar dispositivos Arduino de forma remota. Puedes crear proyectos de IoT, controlar sensores y actuadores, y visualizar los datos en un panel de control en línea. La solución no es gratuita pero creando una cuenta a partir de un correo personal es posible acceder a una gratuita con las limitaciones propias de toda versión free. Actualmente soporta una gran familia de controladores Arduino, incluida la placa Esp32.

Paso a Paso con Arduino Cloud

Paso 1: Registro y Acceso

Lo primero es ingresar al sitio web de IoT Arduino Cloud y crear una cuenta. Para ello solo es necesario ingresar los datos de un correo electrónico mediante el cual se confirma la creación de la cuenta. También es posible hacer un **sigin** mediante una cuenta asociada de Google. Luego se inicia sesión en la cuenta de IoT Arduino Cloud con las credenciales proporcionadas.

Paso 2: Crear un Nuevo Dispositivo

Después de iniciar sesión, se ingresa al panel de control y se selecciona la solapa **Dispositivos**. Tener presente que para la plataforma el término dispositivo hace referencia específicamente a los microntroladores. Mediante la opción "ADD" se permite configurar un nuevo dispositivo. Tener en cuenta que la placa Esp32 corresponde a un dispositvo de tercero por lo que deberá seleccionar la opción **Third party device**. Se debe asignar un nombre único al dispositivo y seleccionar el modelo específico de placa (La placa UTN corresponde

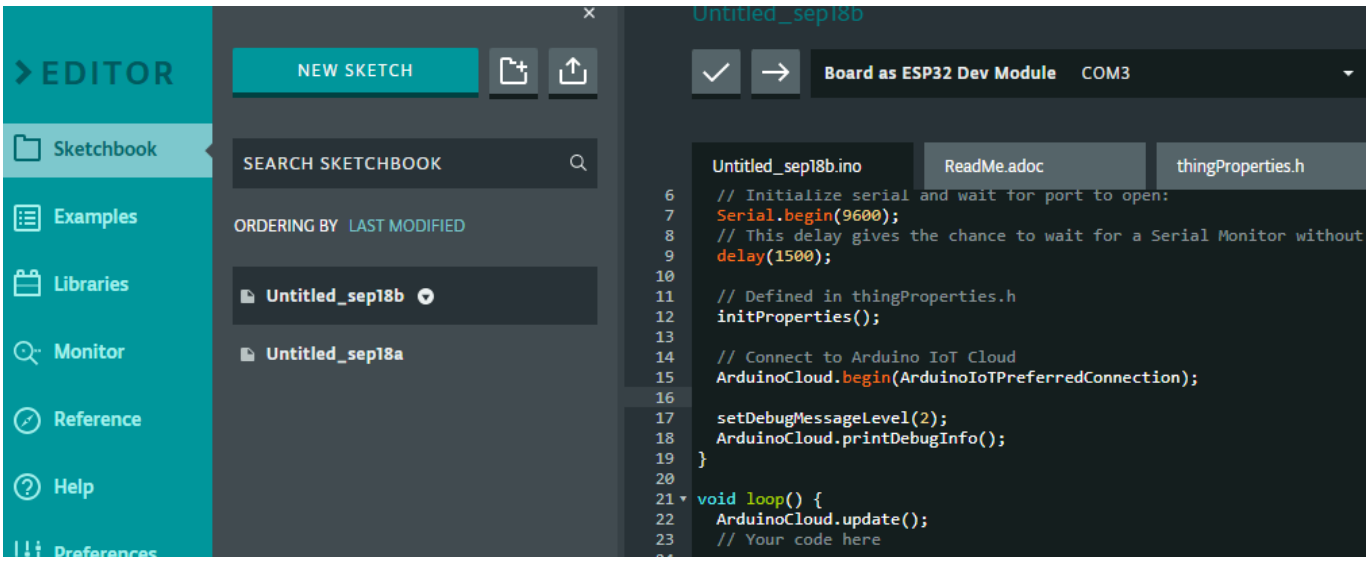
con la opción **ESP32 Dev Module**). Lo más importante de este paso es recordar el ID y la clave de autenticación que se genera. Éstas credenciales se utilizarán para conectar desde el dispositivo físico a la plataforma IoT. Siempre se aconseja descargar estos valores del documento pdf que genera automáticamente.

Paso 3: Arduino IDE para IoT Arduino Cloud

> Desde Arduino IDE (local)

Se debe instalar la biblioteca Arduino IoT Cloud siguiendo estos pasos: a. Desde "Herramientas" > "Gestor de tarjetas". b. Buscar "Arduino IoT Cloud" y seleccionar la opción "Instalar". Abre un nuevo proyecto en el Arduino IDE y selecciona la placa Arduino que estás utilizando. Ve a "Programar" > "Incluir Biblioteca" > "Arduino IoT Cloud" y selecciona tu dispositivo en "Dispositivo Arduino". Pega la clave de autenticación que copiaste en el paso 2 en el código Arduino.

> Desde el IDE remoto



Adicionalmente es posible trabajar directamente desde el sitio ingresando en la opción **sketch**. Actualmente la plataforma tiene una vista de desarrollo completa muy similar al entorno Arduino IDE de escritorio pero para poder utilizarlo correctamente es necesario instalar un pluggin que la propia plataforma sugiere descargar cuando se selecciona esta vista de desarrollo. Algo interesante es notar que el proyecto que genera la plataforma ya se encuentra divido en varios archivos incluidos un archivo de configuración y una librería propia para la declaración y variables y funciones auxiliares.

Paso 4: Crear un Think (Lógica de Control)

Desde el panel de control la opción **Thinks** permite crear** las cosas**. Tener presente que este término está vinculado con actuadores y sensores. Desde la opción "ADD" se crea y se asigna un nombre al nuevo objeto. Si el plan es free solo permite crear dos things como máximo. Luego se configuran las reglas y acciones que se necesita que realice el dispositivo vinculado. Por ejemplo, se puede crear un think que encienda un LED cuando la temperatura supere cierto umbral.

Un concepto interesante en este paso es de **variables**. Las variables son las herramientas con las que es posible monitorear o controlar el thing desde el **dashboard**. Por ejemplo una variable de tipo bool que tome el valor desde un switch dibujado en el panel de control diseñado.

Paso 5: Crear un Dashboard

Para poder interactuar con los things creados en el paso anterior es necesario diseñar un panel (o dashboard). Un dashboard se personaliza mediante widgets como gráficos, botones (switch) o medidores que permiten mostrar los datos de los dispositivos y controlarlos. Luego se Asocian los things que se crearon en el paso 4 a los widgets del dashboard para controlar y mostrar los datos en tiempo real.

Paso 6: Conectar y Monitorear tu Dispositivo

Por último se sube el código Arduino al dispositivo siguiendo las instrucciones habituales del Arduino IDE. Una vez que el dispositivo está en línea, se podrá monitorear y controlar su funcionamiento desde el dashboard que se creó en el paso 5.

También es posible utilizar una app para dispositivos móviles que puede desacargase de las tiendas habituales. Mediante estas app se trabaja con los dashboard diseñado de la misma manera que se haría desde el entorno de trabajo ofrecio por el sitio Web de la plataforma.

Para más detalle se sugiere el siguiente video.

Ejercitación

Caso Práctico 12: Se sugiere diseñar la misma estación meteorológica desarrollada mediante ThingSpeak pero esta vez usando Arduino Cloud.

IOT y MQTT

Fundamentos del Protocolo MQTT para IoT

En el vertiginoso mundo del Internet de las Cosas (IoT), la comunicación eficiente entre máquinas (M2M) es la piedra angular de cualquier solución robusta. Los dispositivos IoT a menudo operan con recursos computacionales limitados y se comunican a través de redes poco fiables. Para superar estos desafíos, MQTT (Message Queuing Telemetry Transport) se ha consolidado como un pilar estratégico para arquitecturas IoT que priorizan la resiliencia operativa y la eficiencia de costos en despliegues a gran escala, proporcionando una solución elegante y eficaz para la transmisión de datos.

La mensajería en el contexto de IoT facilita un flujo de información desacoplado entre una multitud de dispositivos dispares —desde simples sensores hasta actuadores complejos— y los sistemas de back-end. Este enfoque es fundamental para construir arquitecturas escalables y mantenibles. MQTT implementa este paradigma a través del modelo de Publicador-Suscriptor (Pub/Sub). A diferencia de los modelos tradicionales de solicitud-respuesta, el modelo Pub/Sub desacopla a los clientes que producen información (publicadores) de aquellos que la consumen (suscriptores). Los publicadores envían mensajes a un intermediario central, sin necesidad de saber quién, o incluso si alguien, los recibirá. Esta arquitectura no solo simplifica la lógica de los dispositivos, sino que también mejora drásticamente la escalabilidad del sistema global.

Este modelo se sustenta en dos componentes centrales que orquestan todo el flujo de comunicación:

- Broker: Es el servidor intermediario que recibe todos los mensajes de los publicadores y los reenvía a los suscriptores correspondientes. El broker filtra los mensajes entrantes basándose en "temas" y se asegura de que solo los clientes suscritos a un tema específico reciban los mensajes publicados en él. Un ejemplo concreto y ampliamente utilizado de un broker MQTT es Mosquitto, una implementación

de código abierto. Desde una perspectiva de arquitectura, si bien el broker simplifica la comunicación, también representa un componente crítico. Por ello, en despliegues de producción, es imperativo implementar estrategias de alta disponibilidad y tolerancia a fallos para el broker.

- Topics (Temas): Actúan como un sistema de direccionamiento o un canal para los mensajes. Los publicadores envían mensajes a temas específicos, y los suscriptores reciben mensajes al registrar su interés en esos mismos temas. Por ejemplo, un sensor de temperatura podría publicar sus lecturas en el tema hogar/sala/temperatura. Este mecanismo permite un filtrado granular y asegura que los dispositivos solo reciban la información relevante. Además, MQTT soporta el uso de comodines (wildcards) en las suscripciones, permitiendo a un cliente suscribirse a múltiples temas a la vez (ej. hogar/sala/+ para todos los sensores de la sala), lo que simplifica la lógica y aumenta la flexibilidad del sistema.

Una vez comprendida la arquitectura del protocolo, es esencial analizar cómo se estructuran y formatean los datos que viajan a través de estos canales, lo que nos lleva a la implementación práctica en dispositivos reales.

Implementación Práctica: ESP32 como Dispositivo IoT con MQTT

Pasar de la teoría a la práctica es un paso fundamental para materializar el potencial del IoT. La combinación de un microcontrolador potente y de bajo costo como el ESP32, un formato de datos estándar como JSON y el protocolo MQTT permite crear soluciones de IoT robustas, interoperables y funcionales. El ESP32 ofrece la conectividad WiFi y la capacidad de procesamiento necesarias, mientras que JSON proporciona una forma ligera y legible de estructurar los datos, y MQTT se encarga de transportarlos de manera eficiente.

El formato JSON (JavaScript Object Notation) juega un papel crucial en la comunicación IoT. Su naturaleza ligera, basada en texto y fácilmente legible tanto por humanos como por máquinas, lo convierte en una opción ideal para estructurar las cargas útiles (payloads) de datos de los sensores. En el desarrollo para sistemas embebidos, existen librerías especializadas como cJSON que permiten analizar y generar cadenas JSON de manera eficiente. Esto facilita la integración con plataformas y servicios de back-end, que a menudo exponen APIs de gestión basadas en REST y esperan cargas útiles de telemetría en formato JSON a través de MQTT.

Para ilustrar este proceso, consideremos un caso práctico conceptual que describe el flujo completo de datos, desde la captura en un sensor hasta su publicación en un broker MQTT.

- hardware: El escenario comienza con un microcontrolador ESP32, que actúa como el cerebro del dispositivo. Conectado a él, tenemos un sensor de presión barométrica como el BMP180. La comunicación entre el ESP32 y el sensor se establece a través del bus I2C. Este escenario se basa en configuraciones de hardware documentadas, donde los pines SCL y SDA del sensor se conectan directamente a los pines GPIO correspondientes del ESP32, como GPIO22 y GPIO21 respectivamente, que son los predeterminados para el bus I2C.
- Software: El software que se ejecuta en el ESP32 sigue una secuencia lógica clara. Primero, utiliza las librerías del controlador I2C para leer los datos brutos del sensor BMP180. A continuación, estos datos se formatean en una cadena de texto JSON utilizando una librería como cJSON. El resultado podría ser una carga útil como {"presion": 1013.25, "temperatura": 22.5}. Finalmente, el ESP32 emplea una librería de cliente MQTT, como la que puede proporcionar el framework Mongoose, para publicar este mensaje JSON en un tema específico (por ejemplo, sensores/BMP180/datos) en el broker MQTT.
- Comunicación: Para que la publicación sea posible, el ESP32 utiliza su capacidad de conectividad WiFi integrada para establecer una conexión de red con el broker MQTT. Una vez conectado al broker, el

cliente MQTT en el ESP32 envía el mensaje JSON, completando así el ciclo de transmisión de datos desde el mundo físico al digital.

Ejercitación

Caso Práctico 13:

Desarrollar un programa para un microcontrolador ESP32 que permita encender y apagar remotamente dos salidas digitales utilizando el protocolo de mensajería MQTT a través de una conexión Wi-Fi.

Especificaciones Técnicas:

Conectividad de Red:

El dispositivo debe conectarse a una red Wi-Fi con el SSID y la contraseña correspondiente.

Mostrar por puerto serie (a 115200 baudios) el estado de la conexión y la dirección IP asignada.

Configuración MQTT:

Utilizar el broker público de Mosquitto: test.mosquitto.org a través del puerto 1883.

El ESP32 debe suscribirse al tema (topic): test/command.

El ID del cliente debe ser único, generado a partir de la dirección MAC del dispositivo (ej. ESP32Client-[MAC]).

Hardware:

Configurar los pines GPIO 26 y GPIO 27 como salidas digitales.

Ambas salidas deben inicializarse en estado BAJO (LOW) al arrancar el dispositivo.

Lógica de Control (Callback):

El sistema debe escuchar los mensajes entrantes en el tema suscrito y actuar según los siguientes comandos de texto exactos:

"ON26": Pone el pin 26 en estado ALTO (HIGH).

"OFF26": Pone el pin 26 en estado BAJO (LOW).

"ON27": Pone el pin 27 en estado ALTO (HIGH).

"OFF27": Pone el pin 27 en estado BAJO (LOW).

Imprimir por el puerto serie el comando recibido con el formato: In: [comando].

Manejo de Errores:

Implementar una rutina de reconexión automática (reconnect) que intente volver a conectarse al broker MQTT cada 5 segundos en caso de perder la conexión.

El código del proyecto se encuentra en Práctico 13